LAB 9

CTC, Beam Search, and Greedy Search.

TAs:
Madhavi (mgulavan@andrew.cmu.edu)
Kangping (kangpinl@andrew.cmu.edu)

Disclaimer

Content of slide borrows from Mengchun and Michael’s slide for lab 9

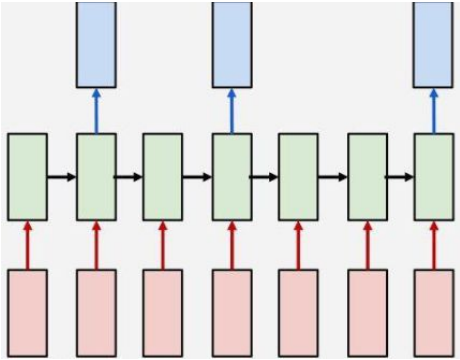
HW1P2 vs. HW3P2

HW1P2

- Sequence Classification for phoneme recognition.
- **Time-synchronous** outputs.

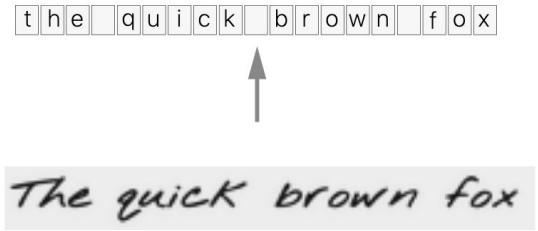
HW3P2

- Sequence to Sequence with **Order Synchrony**
- Training: Using CTC Loss to deal with...
  - The problem of alignment
  - The problem of repetition
- Inference
  - Greedy Search
  - Beam Search

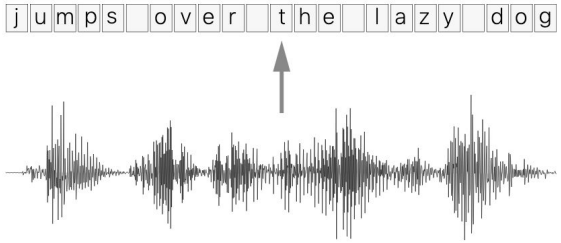


Problem Set-up

- Inputs and targets are not time-aligned
  - |Y| ≠ |X|
  - |Y| and |X| not proportional
- However, they are order-aligned
- We will compress predicted sequences
- We need to be able to yield repeating outputs after compression



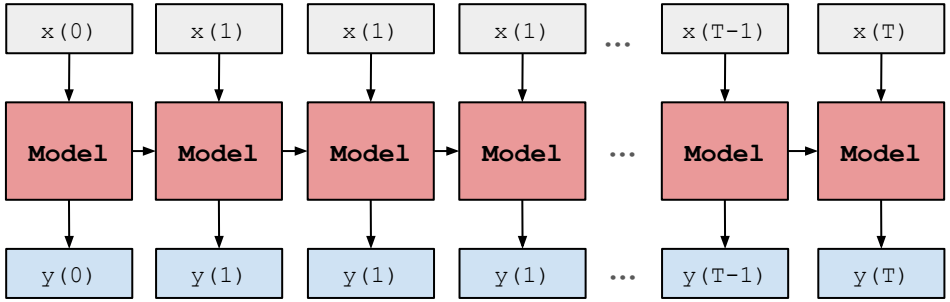
Handwriting recognition: The input can be (x, y) coordinates of a pen stroke or pixels in an image.



Speech recognition: The input can be a spectrogram or some other frequency based feature extractor.

(Training)

Someone says "zoo"



Inputs

Probabilities

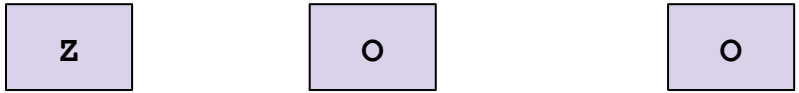
Two problems

(1) time alignment and (2) repetition

One Asset

(1) order alignment

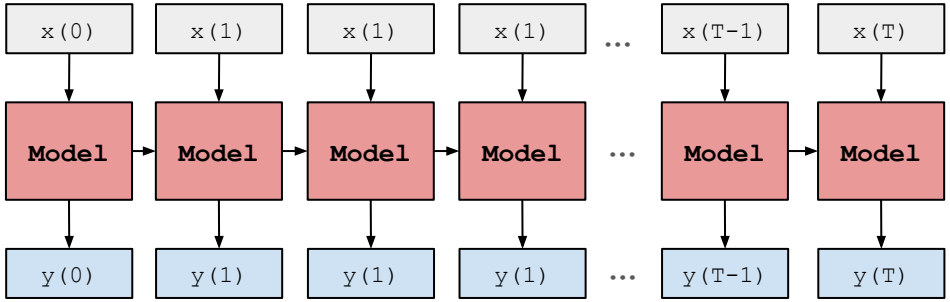
We want to transcribe "zoo" at these time steps



Target

(Training)

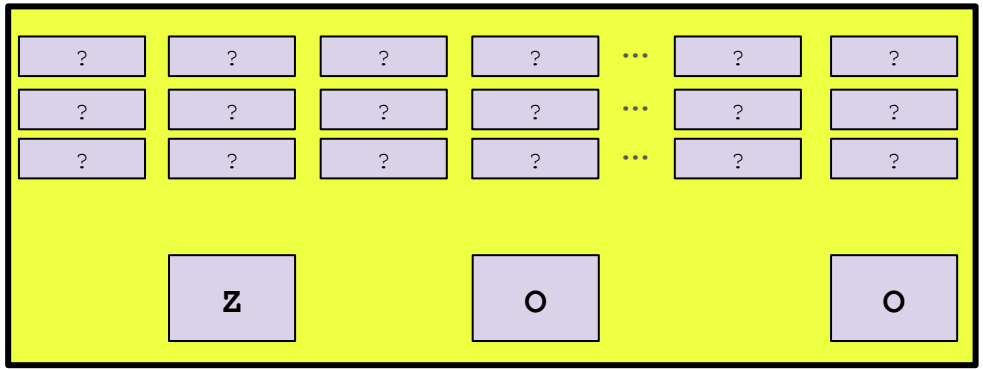
Someone says "zoo"



Inputs

Probabilities

Use **CTC** to get time-aligned targets



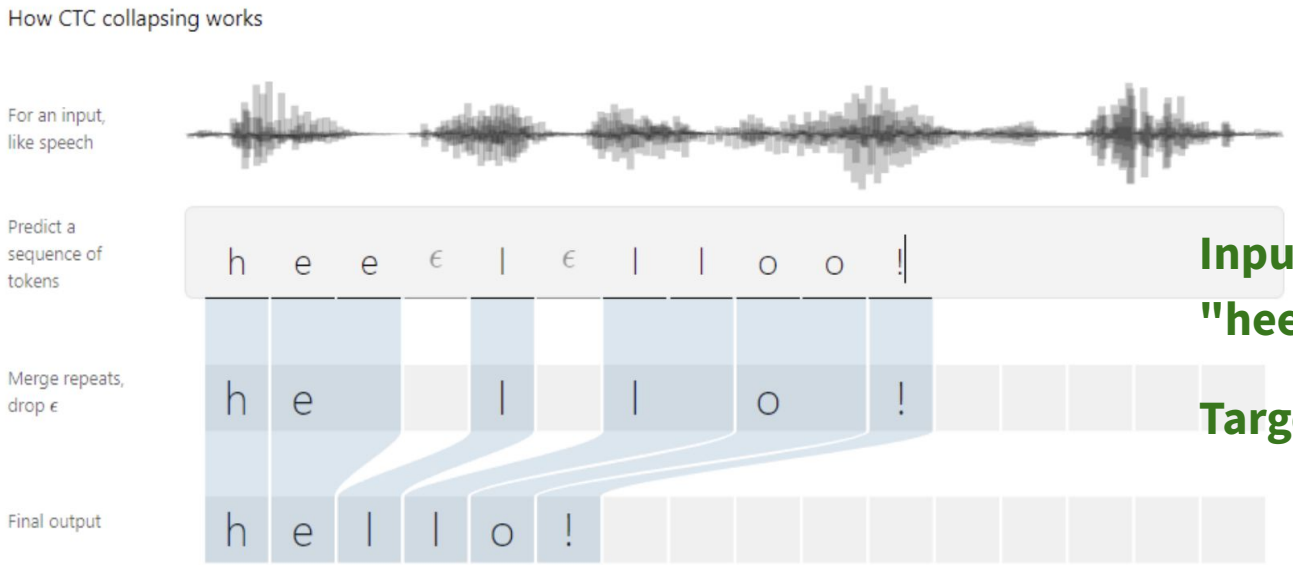
<u>All</u> time-aligned targets for Target

We want to transcribe "zoo" at these time steps

Target

Why CTC?

- CTC is used to calculate loss when our input sequences and output labels have **different lengths and no fixed alignment.**
- This method allows models to learn without needing pre-aligned data, which is crucial for applications like **speech recognition.**



Input (from RNN over time) → "heeϵlϵlloo" (length = 10)

Target sequence → "hello"

The problem of repetition…

A note on repetition…

(Training)

"zzzooo"
"zoo"
"zo"
"zzzzzzzzo"

➡ "zo" (😕)

A note on repetition…

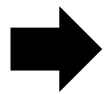
(Training)

"zzzooo"
"zoo"
"zo"
"zzzzzzzzo"

"zo" (🙁)

To account for repetitions, **introduce a "break" character ("%")**

"zzz%oo%o"
"zo%o"
~~"zo"~~
~~"zzzzzzzzo"~~



"zoo"
(😀)

The problem of alignment…

A note on alignment…

(Training)

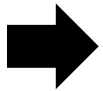
"z%o%oo"
"z%oo%o"
…
"zooo%o"

➡ "zoo" (🙁)

A note on alignment…

(Training)

"z%o%oo"
"z%oo%o"
…
"zooo%o"



"zoo" (🙁)

If we use a time-aligned target, we can compute a differentiable loss.
But which alignment should we choose?

We *could* use the Viterbi algorithm to find the most probable path and use that to calculate the loss. However, there is a better option...

Using the asset of **order alignment**, as well as some **additional "rules"**, we can actually use **ALL POSSIBLE ALIGNMENTS and calculate an EXPECTED LOSS over them**.

We could use the Viterbi algorithm to find the most probable path and use that to calculate the loss. However, there is a better option...

Using the asset of **order alignment**, as well as some **additional "rules"**, we can actually use **ALL POSSIBLE ALIGNMENTS and calculate an EXPECTED LOSS over them**.

1. Alignments b/t X and Y are **monotonic** (once you advance to the next input, you may only keep the corresponding output or advance to the next output)

2. Alignment of X to Y is **many-to-one** (there may be many input elements aligning to a single output element).

3. **Length of Y ≤ Length of X**

We could use the Viterbi algorithm to find the most probable path and use that to calculate the loss. However, there is a better option…

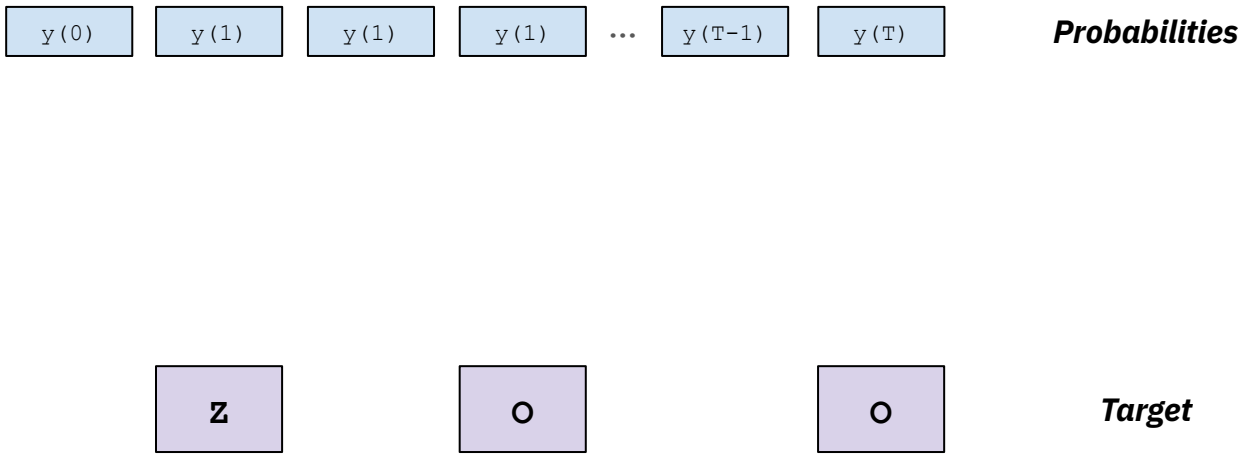
Using the asset of **order alignment**, as well as some **additional "rules"**, we can actually use **ALL POSSIBLE ALIGNMENTS and calculate an EXPECTED LOSS over them**.

⬇

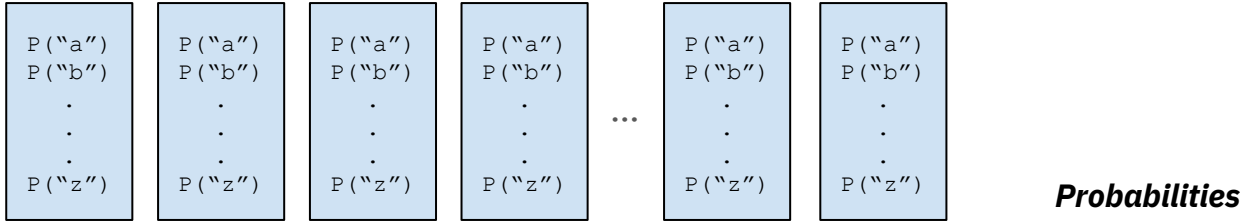
Given: The "break" character, order alignment, and some additional rules…

Task: Find possible alignments, their probabilities, then calculate a differentiable loss

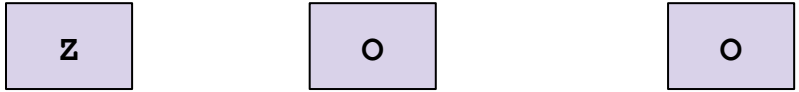
(Training)


y(0)
y(1)
y(1)
y(1)
...
y(T-1)
y(T)
Probabilities
We want to transcribe “zoo” at these time steps
z
o
o
Target

(Training)



We want to transcribe "zoo" at these time steps



(Training)

Limit to the characters in the Target with breaks inserted

P("%")	P("%")	P("%")	P("%")		P("%")	P("%")
P("z")	P("z")	P("z")	P("z")		P("z")	P("z")
P("%")	P("%")	P("%")	P("%")		P("%")	P("%")
P("o")	P("o")	P("o")	P("o")	...	P("o")	P("o")
P("%")	P("%")	P("%")	P("%")		P("%")	P("%")
P("o")	P("o")	P("o")	P("o")		P("o")	P("o")
P("%")	P("%")	P("%")	P("%")		P("%")	P("%")

Probabilities

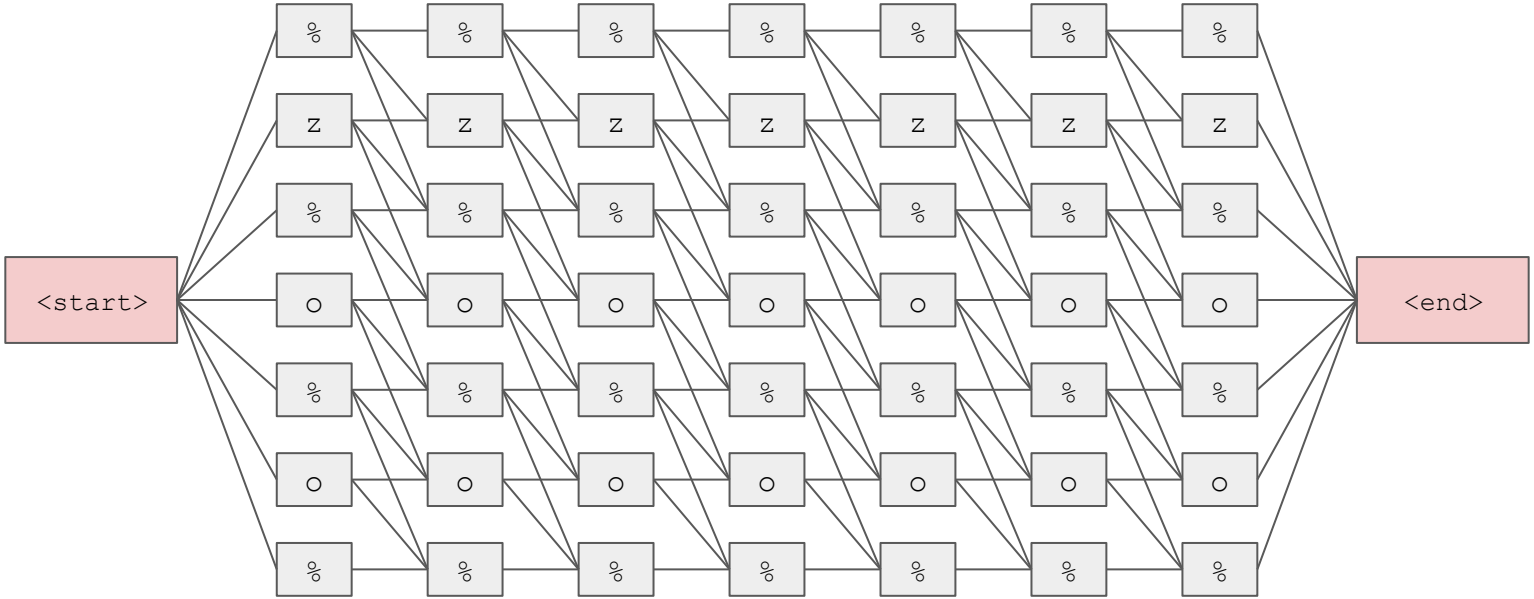
We want to transcribe "zoo" at these time steps

z	o	o

Target

"zoo"

(Training)



Not all of these are "viable" paths/alignments.
Which ones are?

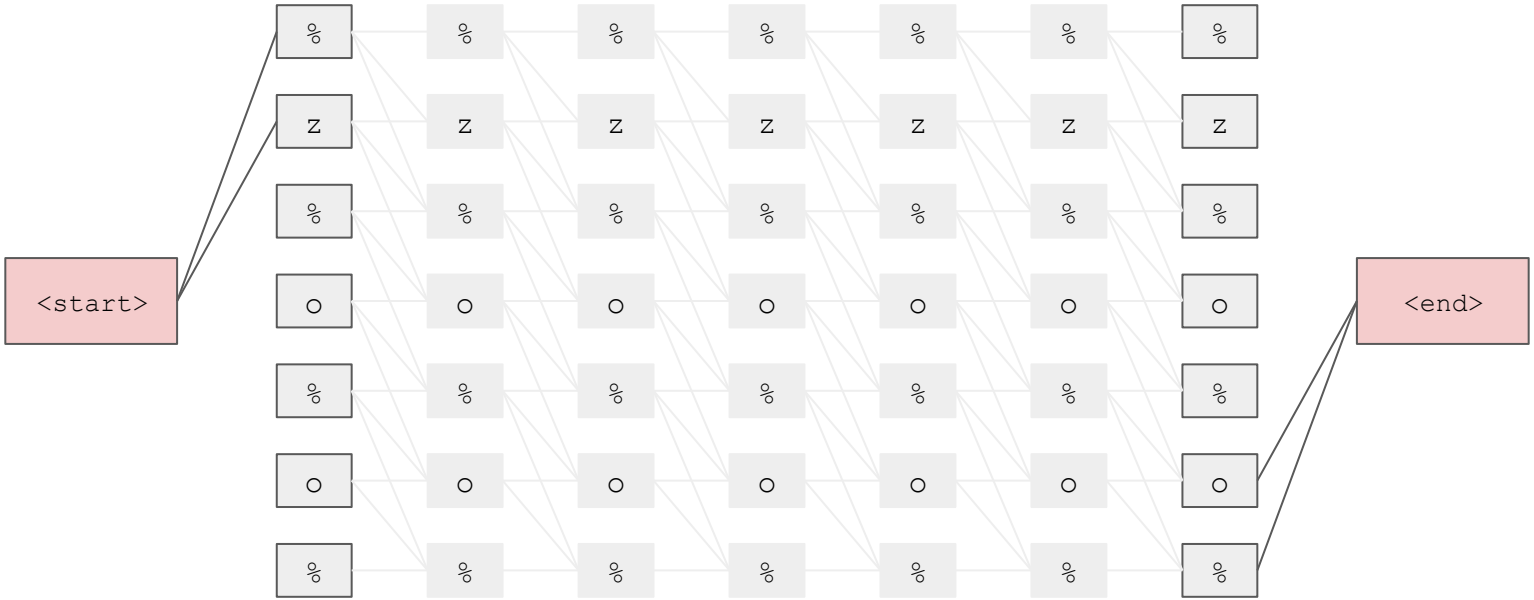
"zoo"

(Training)

Let's start from the outer edges and work our way in...

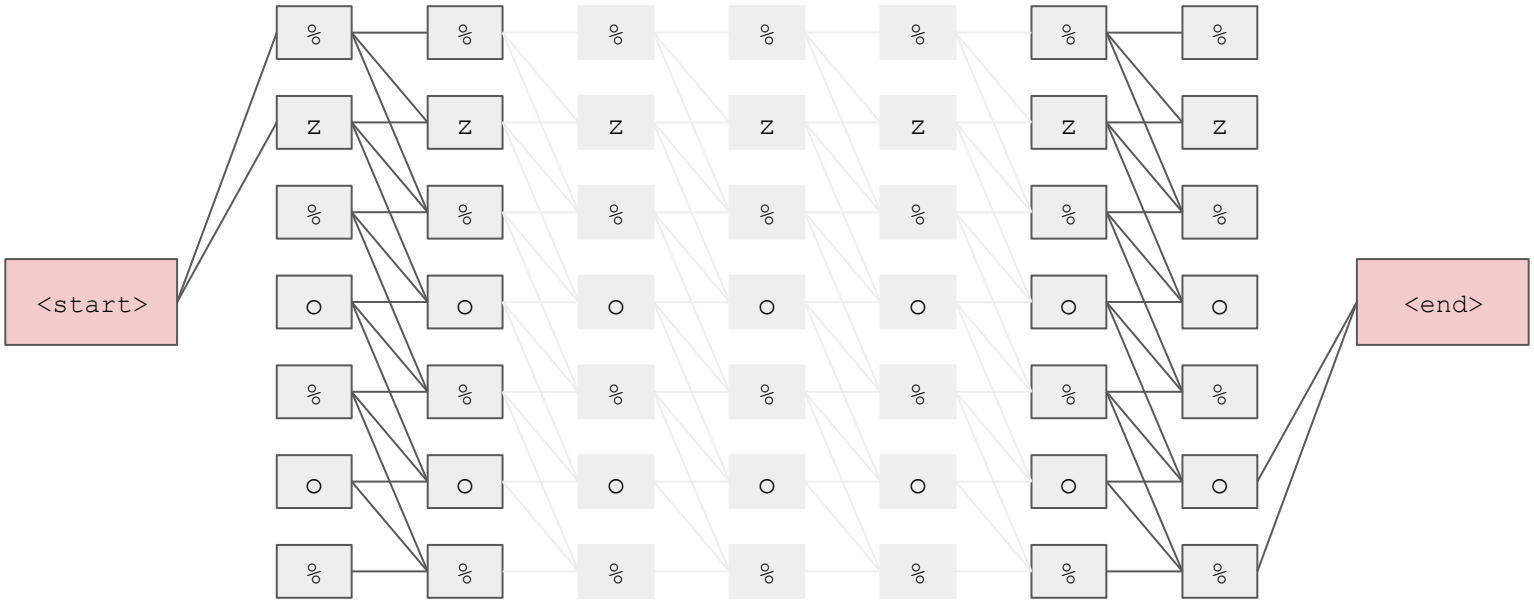
"zoo"

(Training)



Let's start from the outer edges and work our way in...

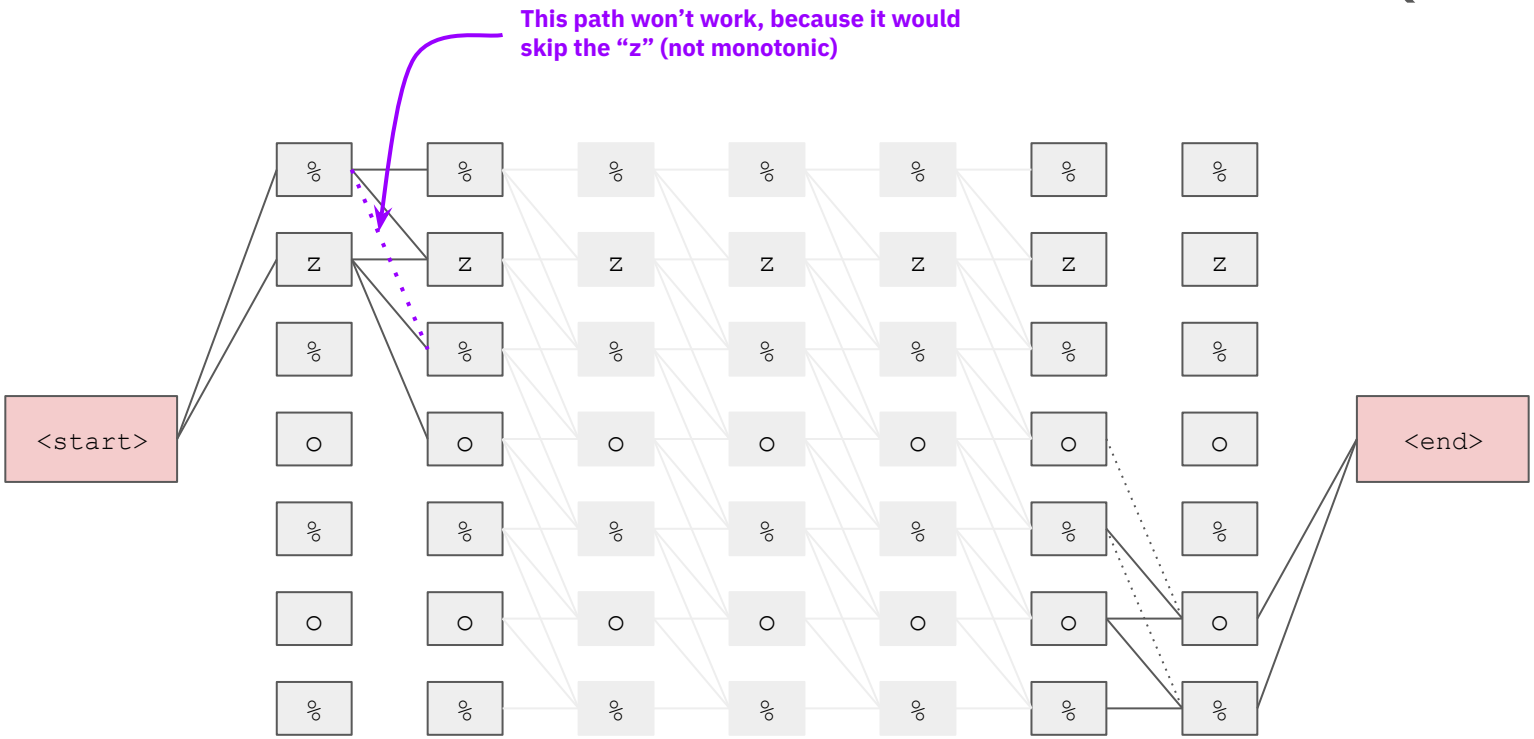
(Training)



Let's start from the outer edges and work our way in...

"zoo"

(Training)

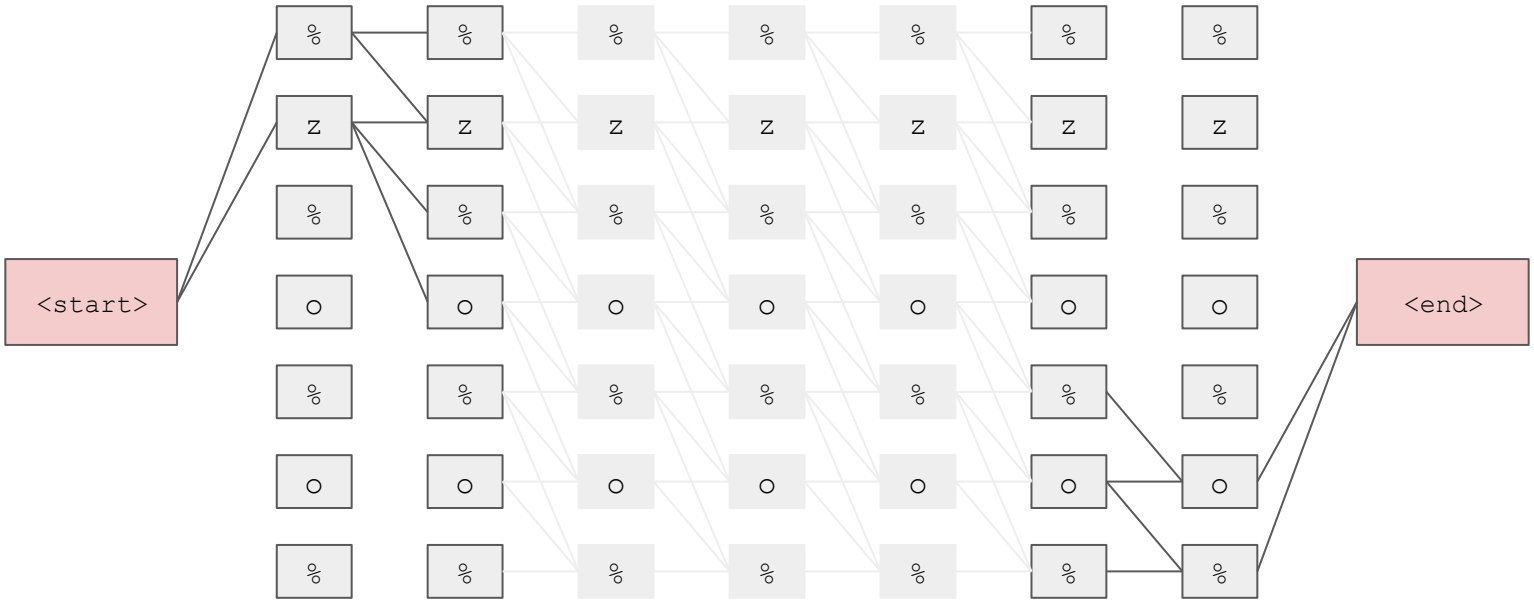


Dotted lines are paths that may seem viable at first, but are not!

Let's start from the outer edges and work our way in...

"zoo"

(Training)



Let's start from the outer edges and work our way in...

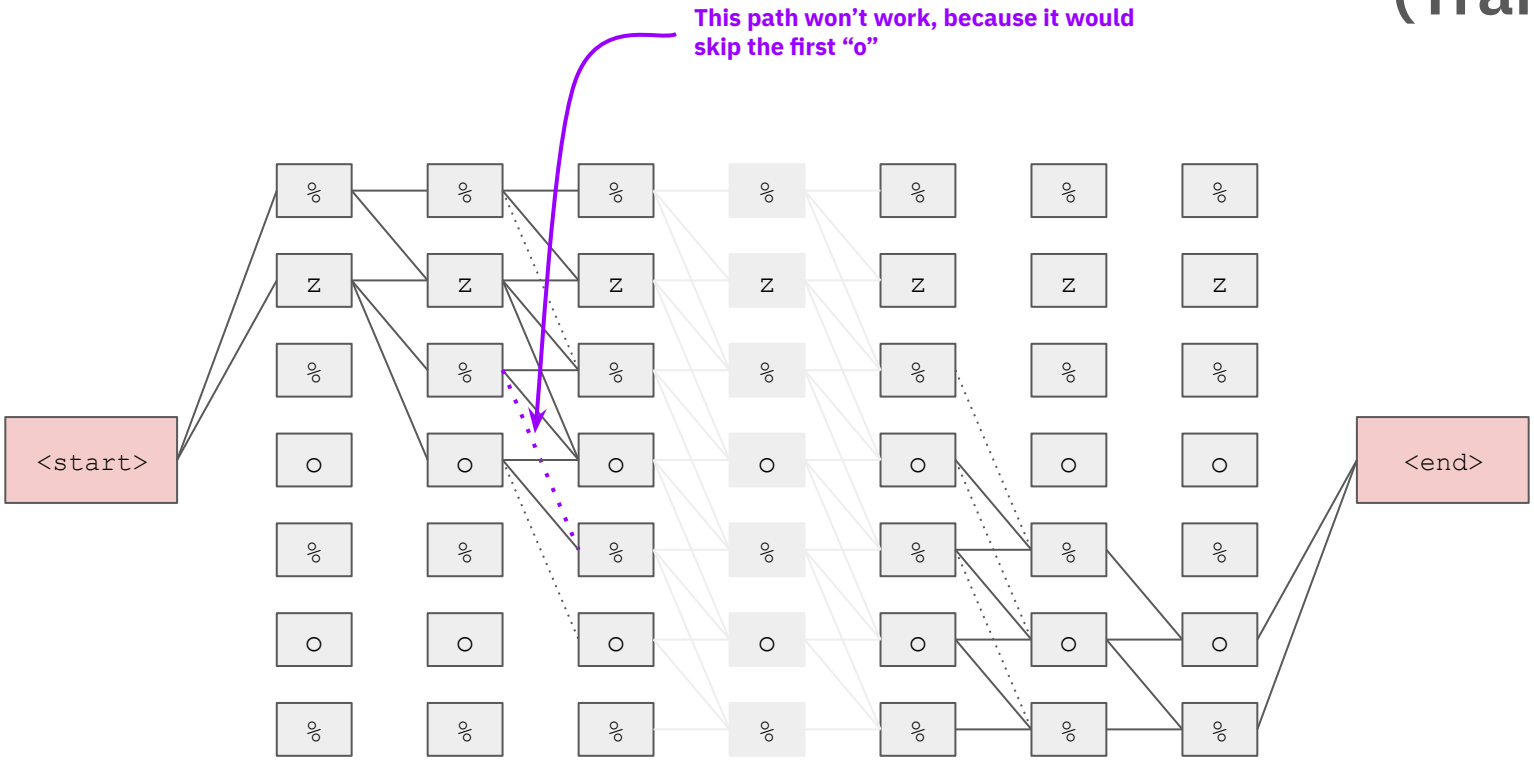
"zoo"

(Training)

Let's start from the outer edges and work our way in...

"zoo"

(Training)



Dotted lines are paths that may seem viable at first, but are not!

Let's start from the outer edges and work our way in...

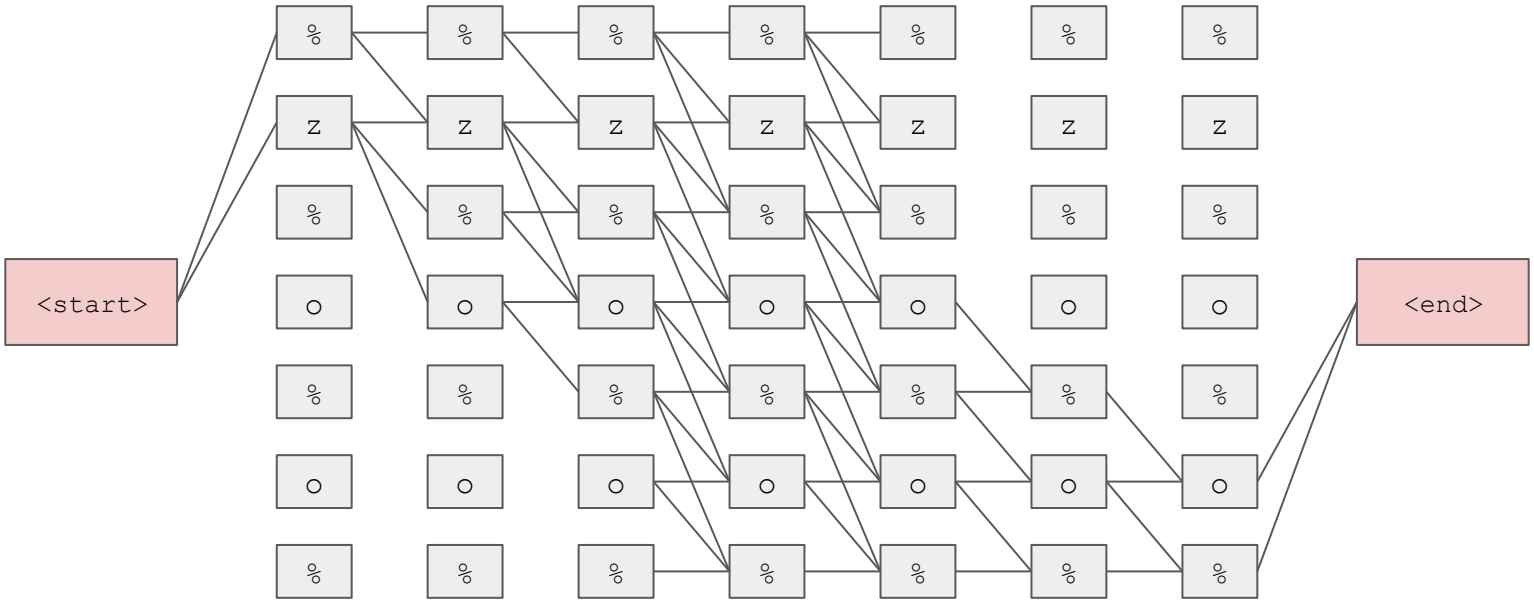
"zoo"

(Training)

Let's start from the outer edges and work our way in...

"zoo"

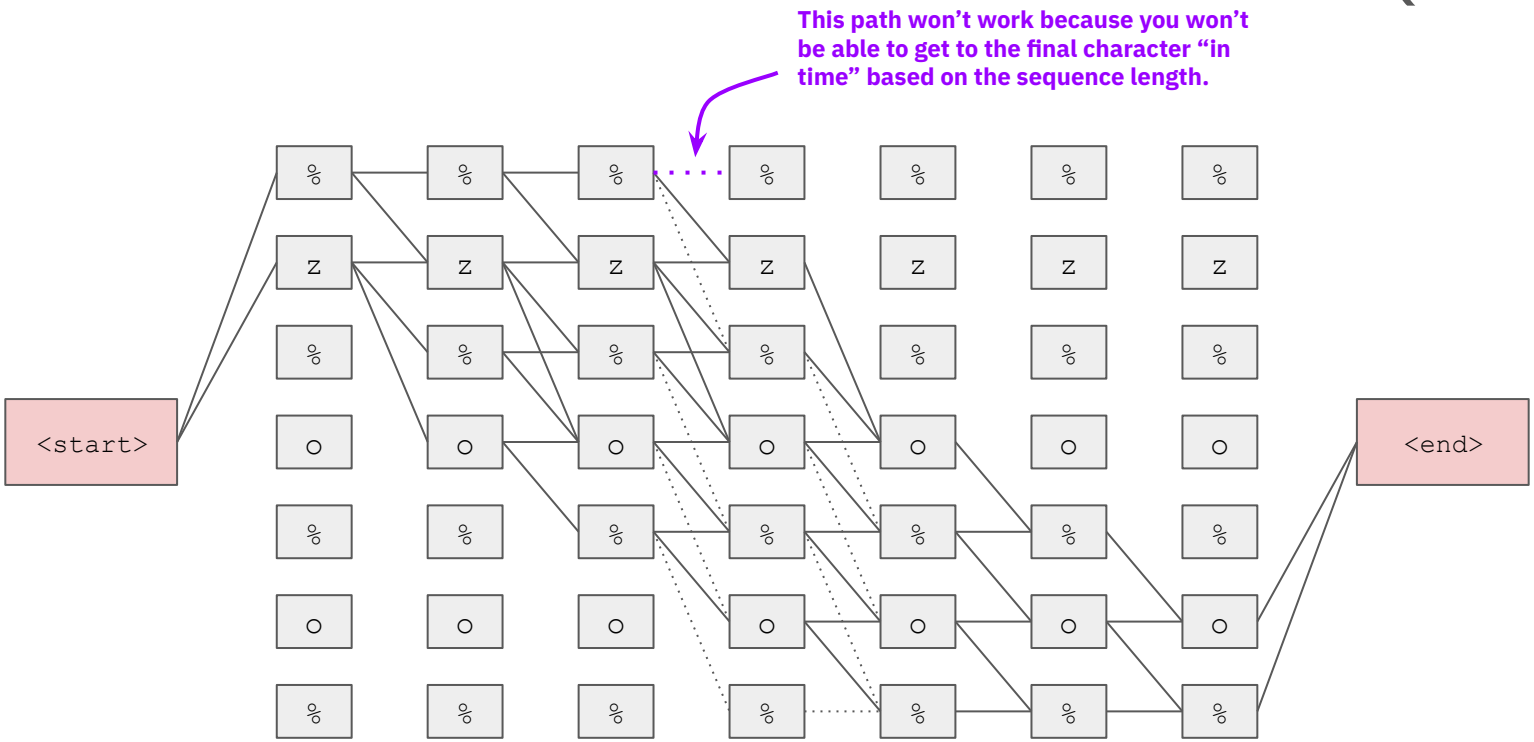
(Training)



Let's start from the outer edges and work our way in...

"zoo"

(Training)

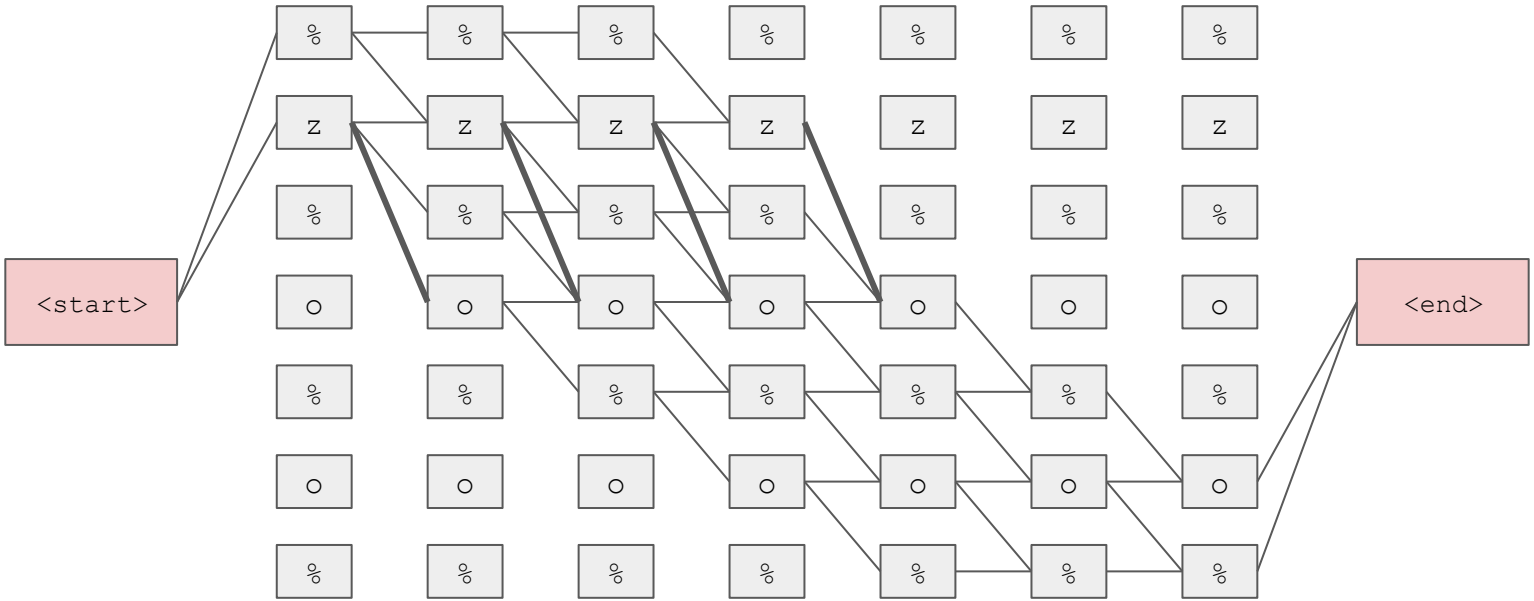


Dotted lines are paths that may seem viable at first, but are not!

Let's start from the outer edges and work our way in...

"zoo"

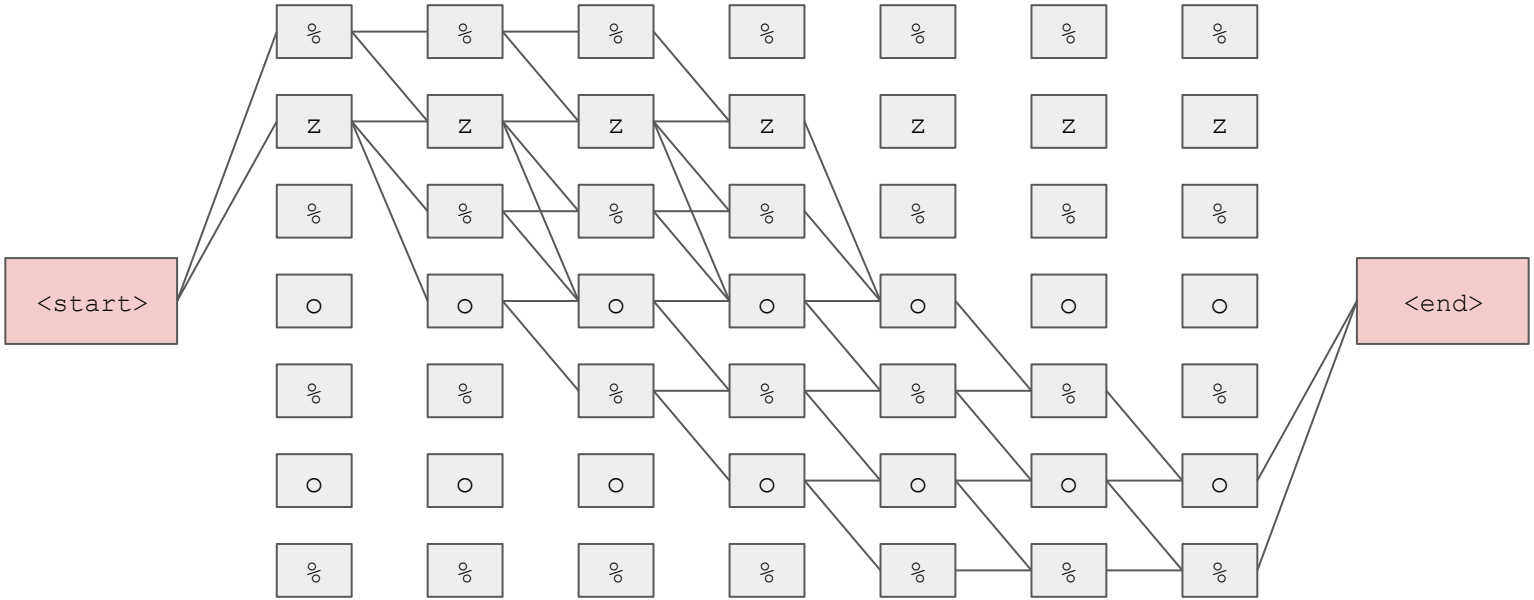
(Training)



These are the viable alignments
(skips bolded)

"zoo"

(Training)



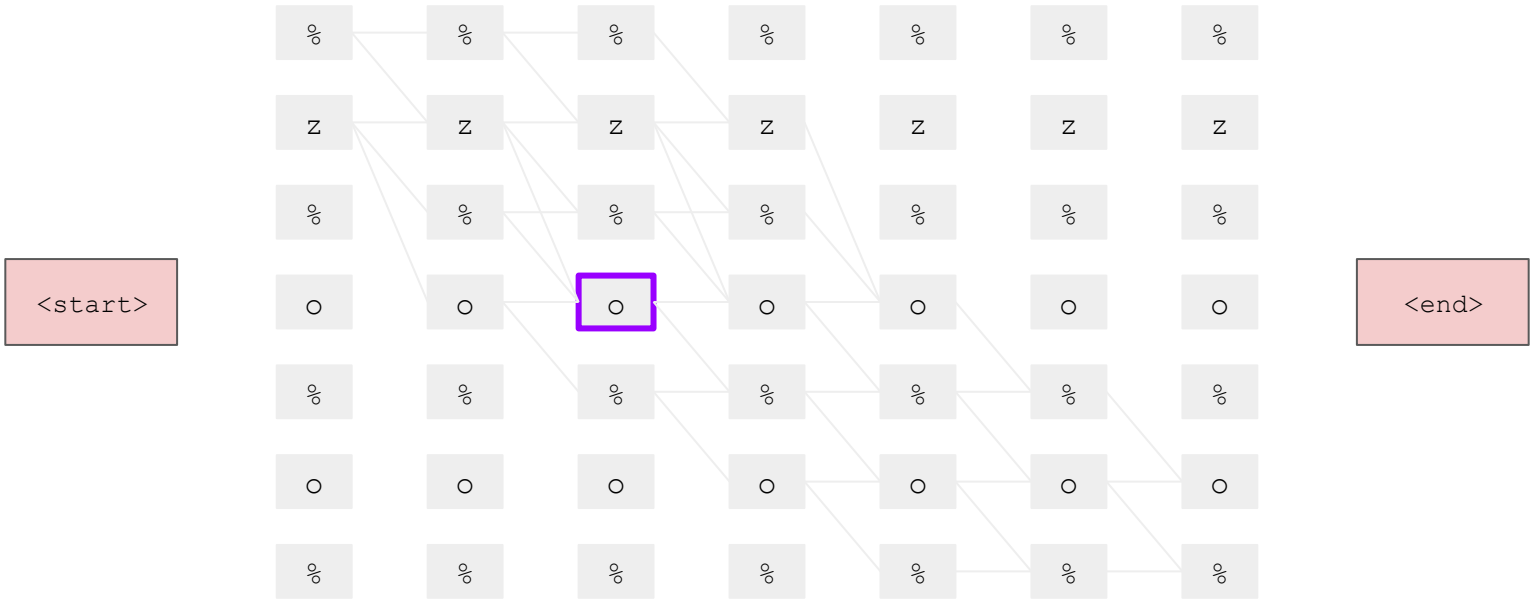
These are the viable alignments

Implementation [CTC/CTC.py]

- CTC
  - extend_target_with_blank()
  - get_forward_probs()
  - get_backward_probs()
  - get_posterior_probs()
- CTCLoss

"zoo"

(Training)

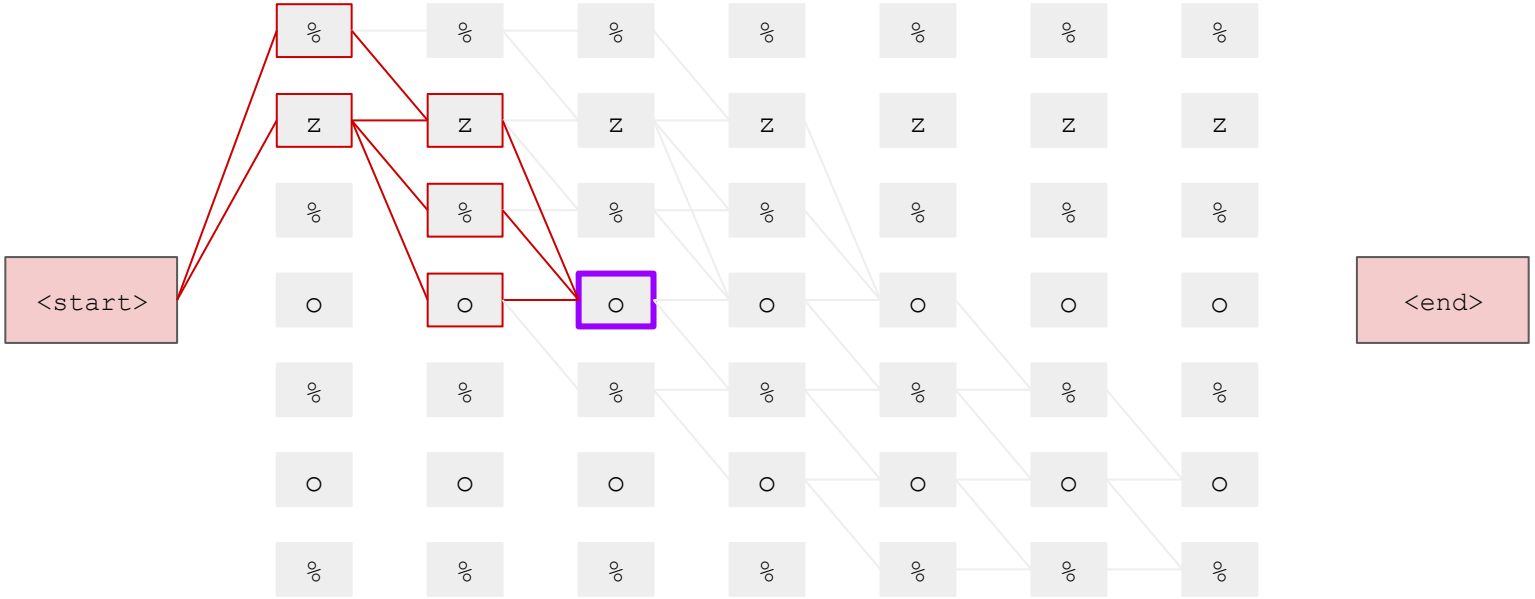


This is where dynamic programming comes in handy!

At any given node, you can calculate the posterior probability of reaching that node at that time step using the product of the probability of reaching from the "**forward**" and "**backward**" passes (See Lecture)

(Training)

Forward Probabilities

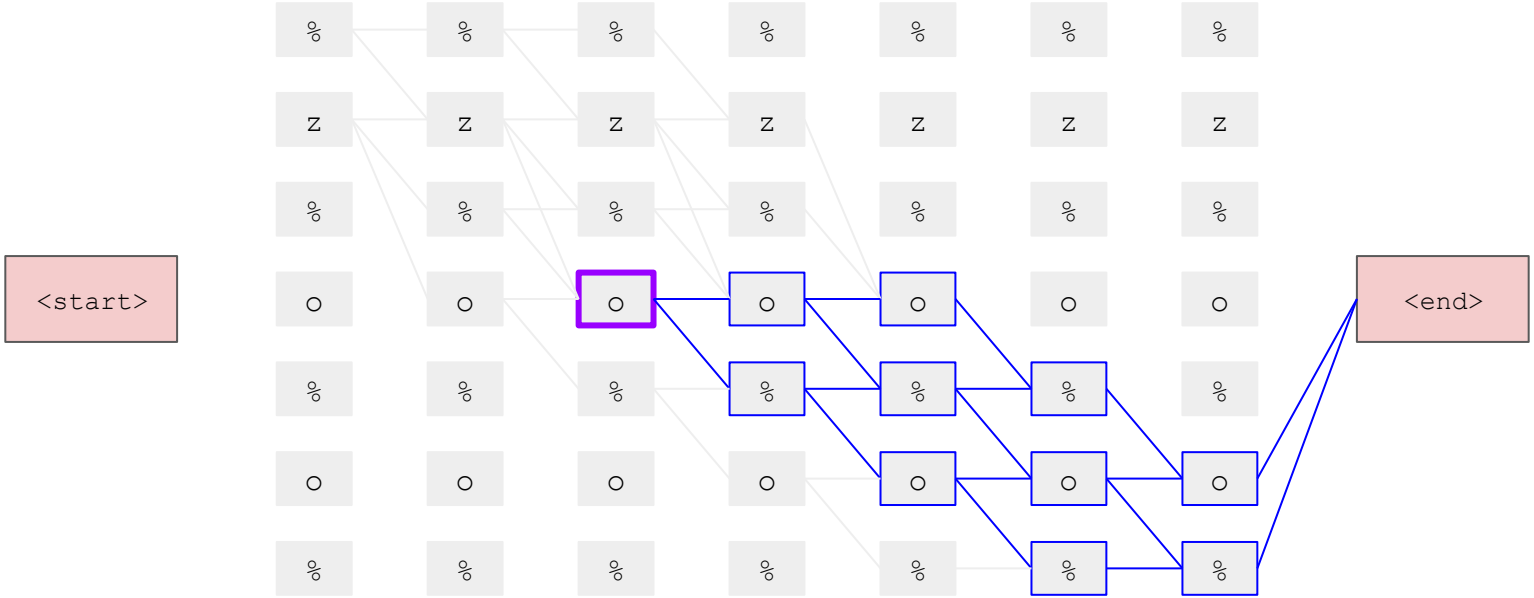


This is where dynamic programming comes in handy!

$$\alpha(t,r) = P(S_0 \ldots S_r, s_t = S_r | X) = \sum_{q:S_q \in pred(S_r)} \alpha(t-1,q) y_t^{S_r}$$

The total probability of all possible paths that start from time 0 and reach symbol r by time t

Backward Probabilities



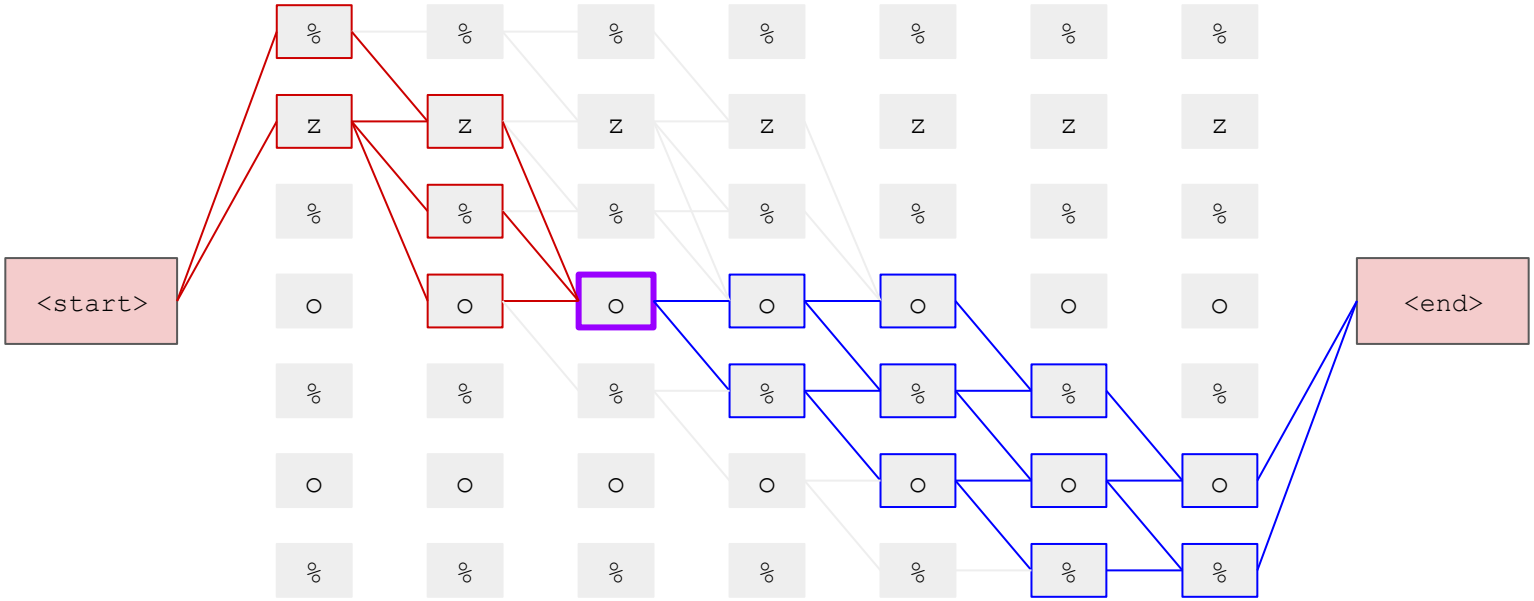
This is where dynamic programming comes in handy!

$$\beta(t,r) = P(s_{t+1} \in succ(S_r), succ(S_r), ..., S_{K-1}|X) = \sum_{q:S_q \in succ(S_r)} \beta(t+1,q) y_{t+1}^{S_q}$$

The probability of generating the remaining part of the sequence starting from time step t at symbol r.

"zoo"

(Training)



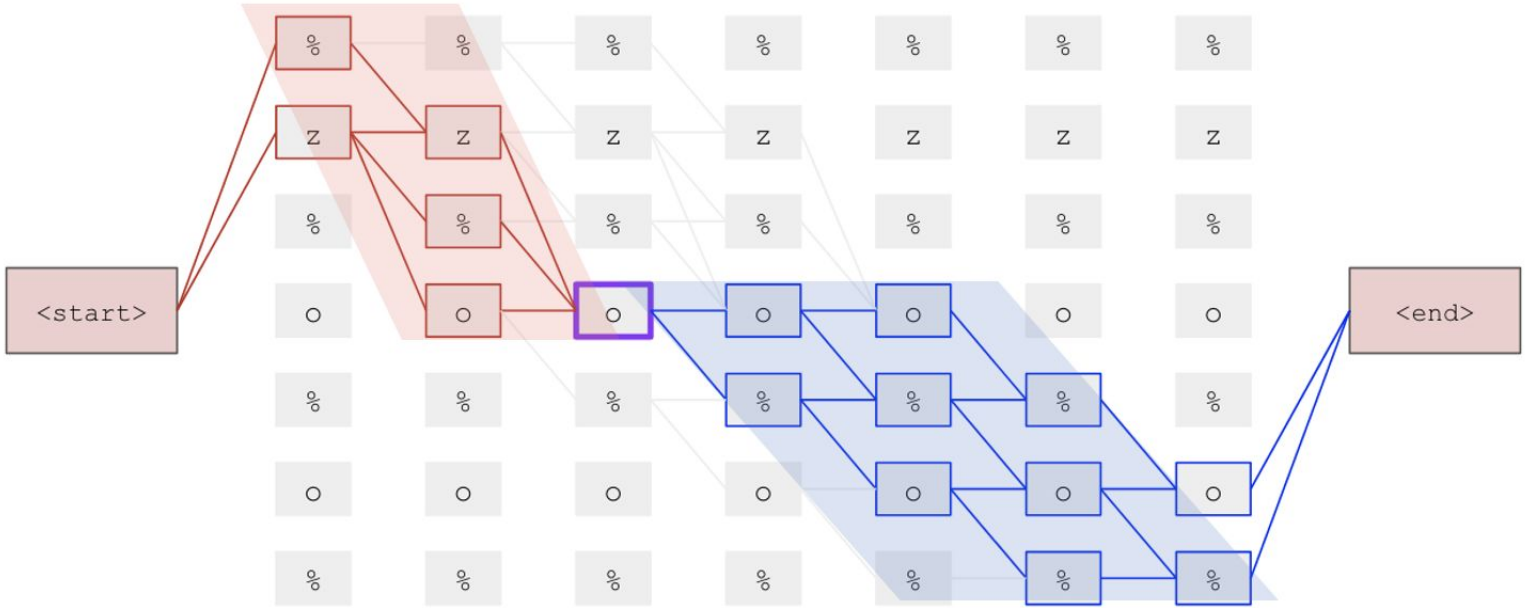
We can calculate the probability of ALL ALIGNMENTS using the forward/backward method and use these to compute an EXPECTED LOSS.

"zoo"

(Training)

Posterior Probabilities:

Represents probability of being in state **r** at time **t** given input sequence.



$$\gamma(t,r) = P(s_t = S_r | S, X) = \frac{\alpha(t,r)\beta(t,r)}{\sum_{r'} \alpha(t,r)\beta(t,r)}$$

CTC Loss

- Once we know the posterior probability, we can plug it in to calculate the loss, which will ultimately be used to train your network.

$$\gamma(t,r) = P(s_t = S_r | S, X) = \frac{\alpha(t,r)\beta(t,r)}{\sum_{r'} \alpha(t,r)\beta(t,r)}$$

$$DIV = -\sum_{t} \sum_{s \in S_0 \ldots S_{K-1}} P(s_t = s | \mathbf{S}, \mathbf{X}) \log Y(t, s_t = s)$$

$$DIV = -\sum_{t} \sum_{r} \gamma(t,r) \log y_t^{S(r)}$$

$$\frac{dDIV}{dy_t^l} = -\frac{1}{y_t^l} \sum_{r:S(r)=l} \gamma(t,r)$$

$$\text{Loss} = -\sum_{\text{all paths}} P(\text{path}|x) \cdot \log P(\text{path output})$$

"zoo"

(Training)

We can calculate the probability of ALL ALIGNMENTS using the forward/backward method and use these to compute an EXPECTED LOSS.

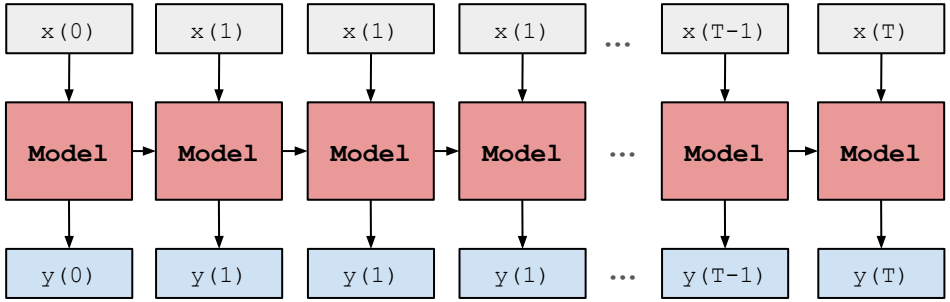
Training Procedure

With Connectionist Temporal Classification(CTC)

1. Define model (e.g., deep bidirectional LSTM)
2. Initialize network to output targets + “break” character
3. Pass training instances through network to obtain probability distribution over labels/symbols
4. Construct graph/table of “viable” alignments
5. Compute probabilities of alignments using Forward/Backward Algorithm (see Lecture)
6. Compute Expected Divergence over all alignments (see Lecture)
7. Propagate gradients backward and update parameters

(Training)

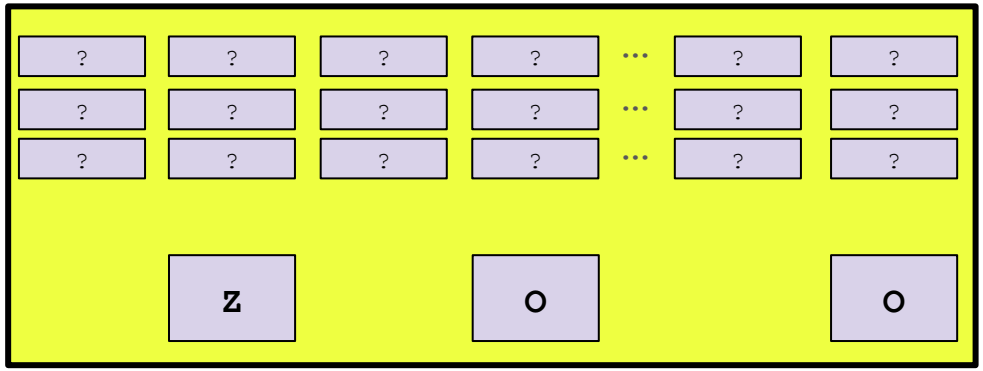
Someone says "zoo"



Inputs

Probabilities

Use **CTC** to get time-aligned targets

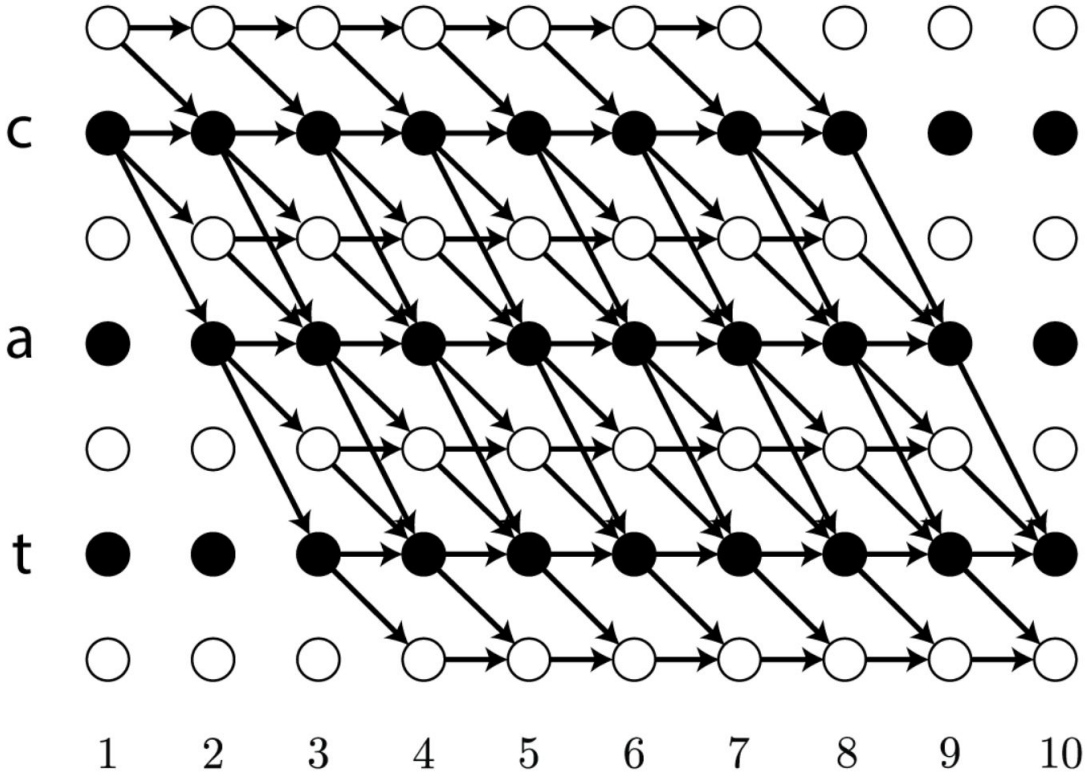


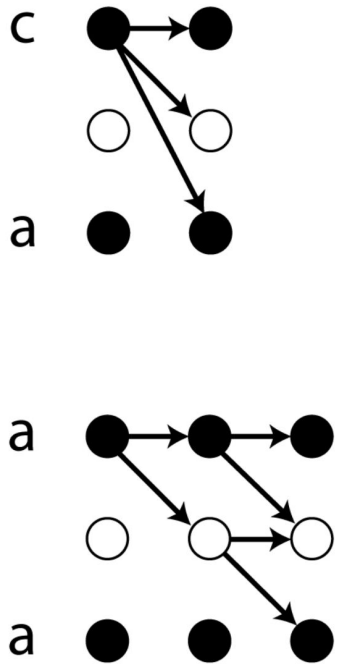
<u>All</u> time-aligned targets for Target

We want to transcribe "zoo" at these time steps

Target

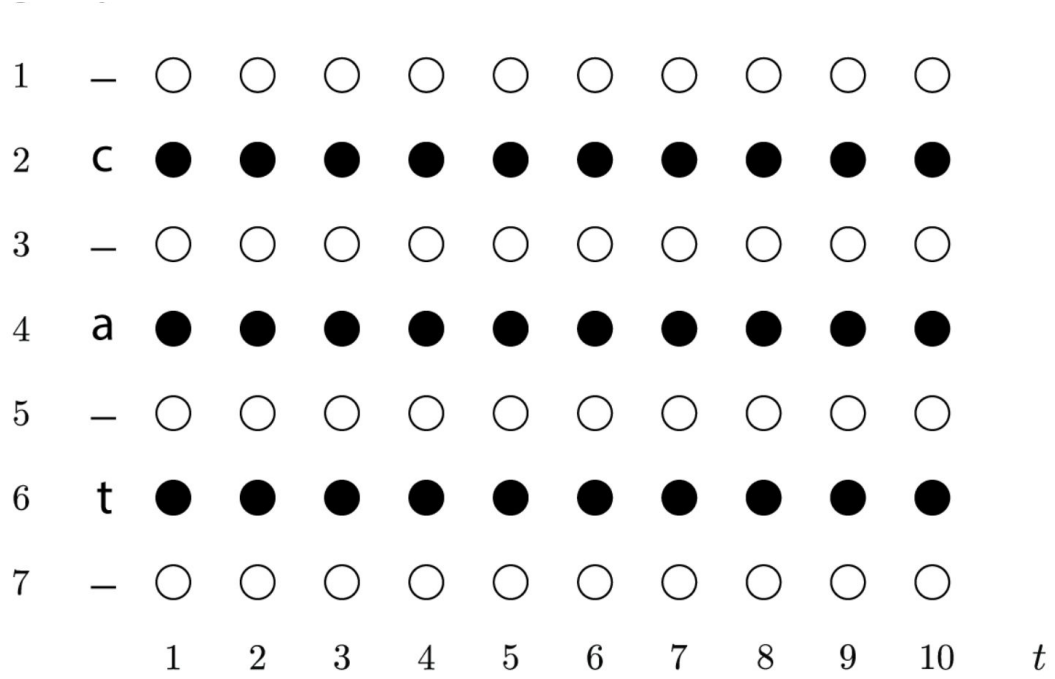
Connectionist Temporal Classification (CTC)





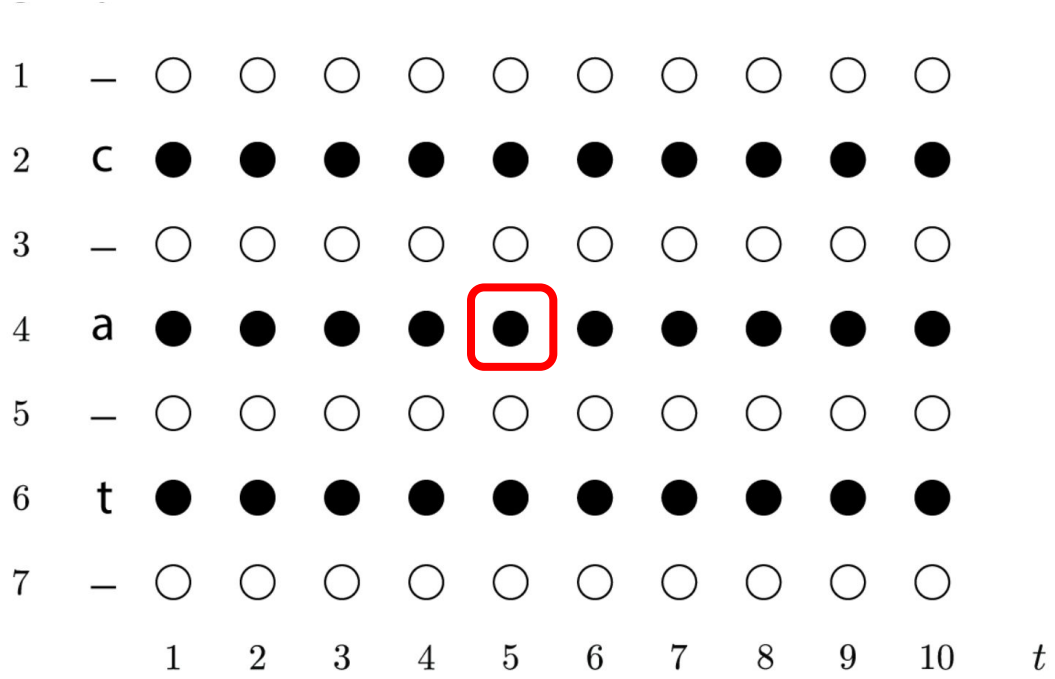
CTC Forward Probabilities

$$\alpha(t,r) = P(S_0 ... S_r, s_t = S_r | X) = \sum_{q:S_q \in pred(S_r)} \alpha(t-1,q) y_t^{S_r}$$



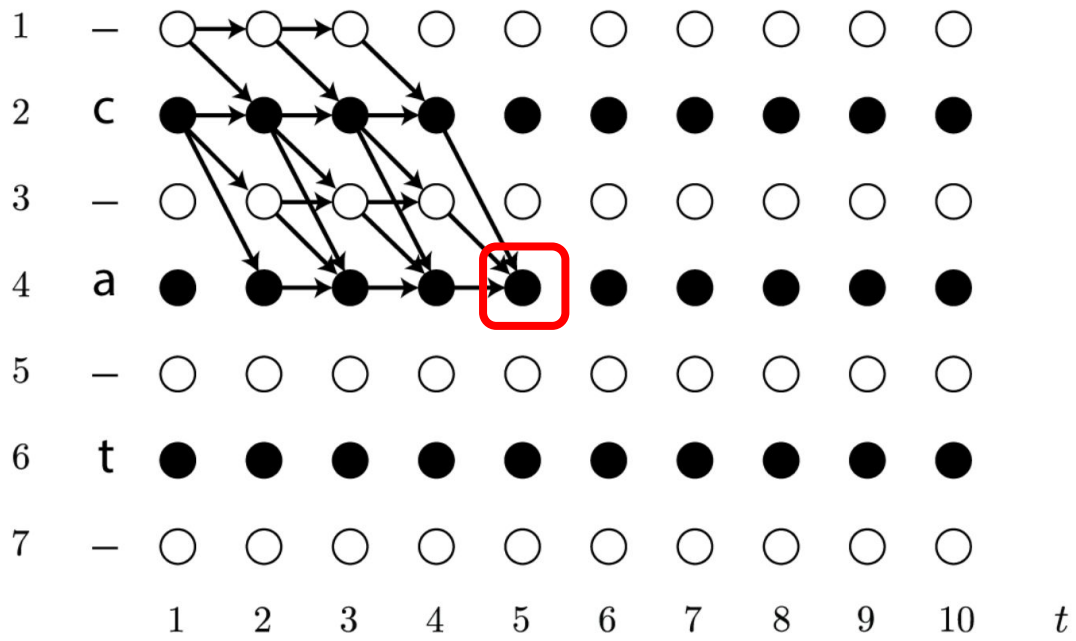
CTC Forward Probabilities

$$\alpha(t,r) = P(S_0 \ldots S_r, s_t = S_r | X) = \sum_{q:S_q \in pred(S_r)} \alpha(t-1,q) y_t^{S_r}$$



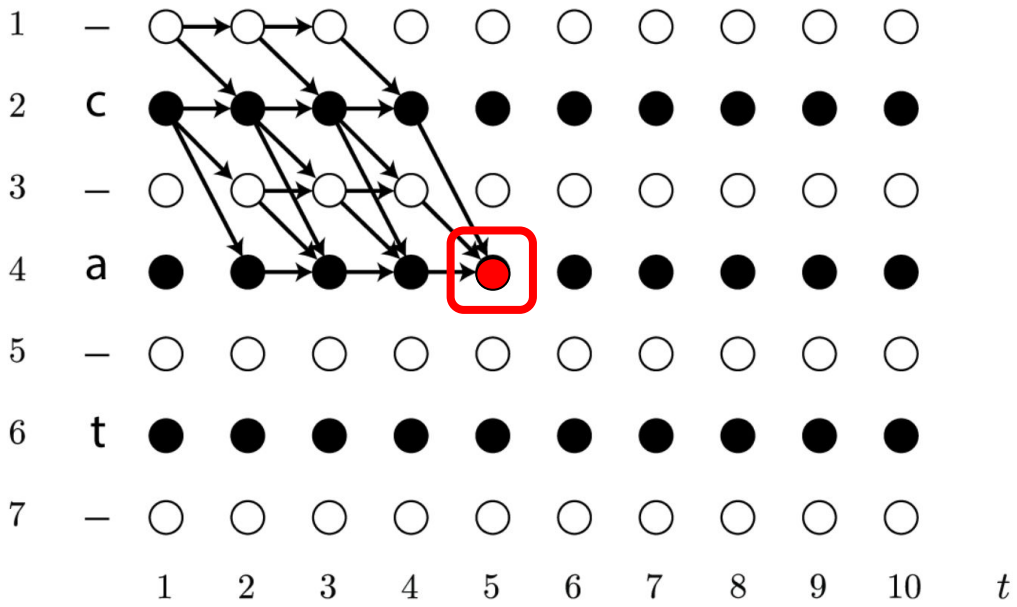
CTC Forward Probabilities

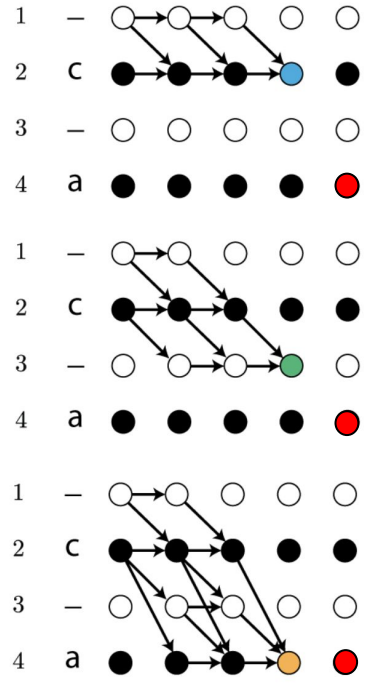
$$\alpha(t,r) = P(S_0 ... S_r, s_t = S_r | X) = \sum_{q:S_q \in pred(S_r)} \alpha(t-1,q) y_t^{S_r}$$

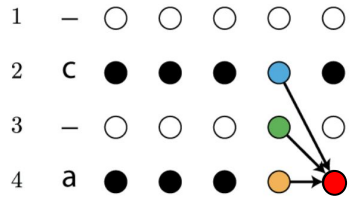


CTC Forward Probabilities

$$\alpha(t,r) = P(S_0 ... S_r, s_t = S_r | X) = \sum_{q:S_q \in pred(S_r)} \alpha(t-1,q) y_t^{S_r}$$

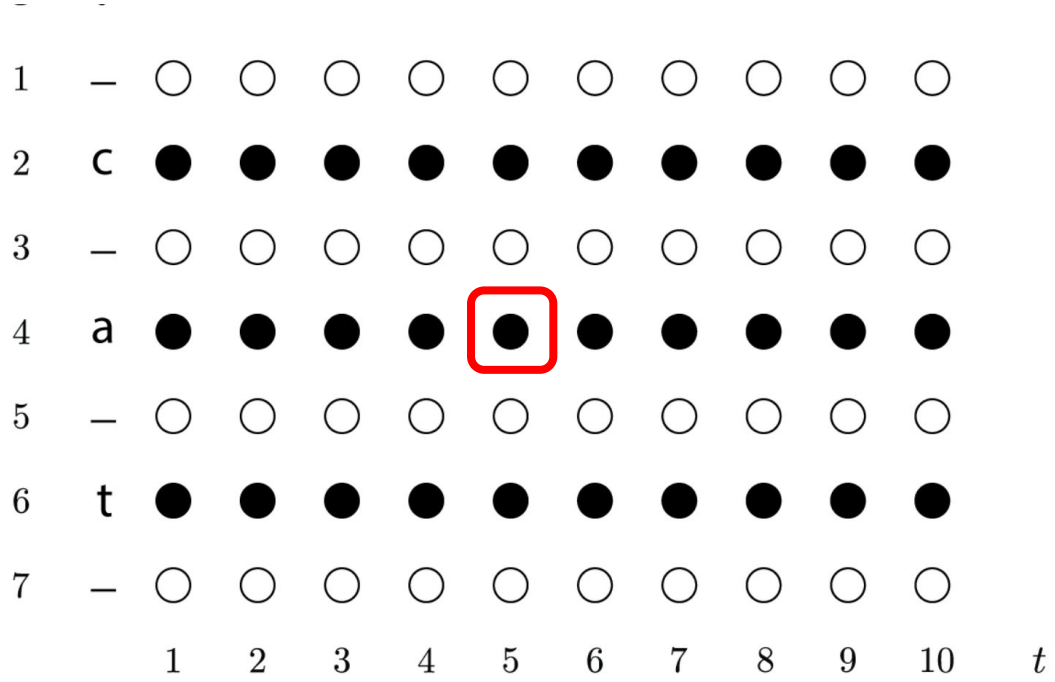






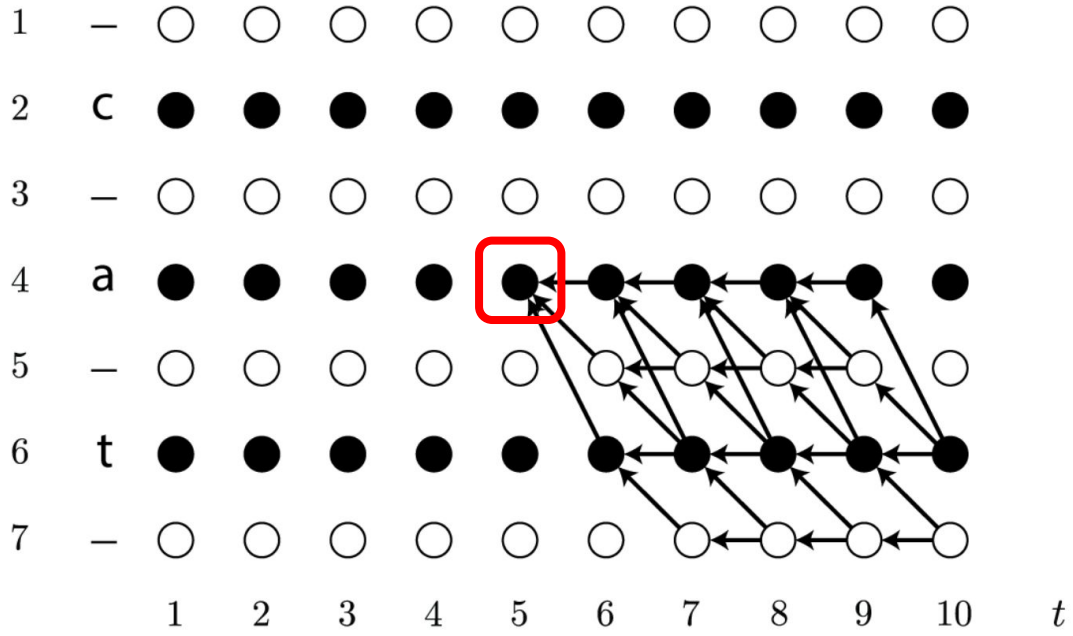
CTC Backward Probabilities

$$\beta(t,r) = P(s_{t+1} \in succ(S_r), succ(S_r), ..., S_{K-1}|X) = \sum_{q:S_q \in succ(S_r)} \beta(t+1,q) y_{t+1}^{S_q}$$



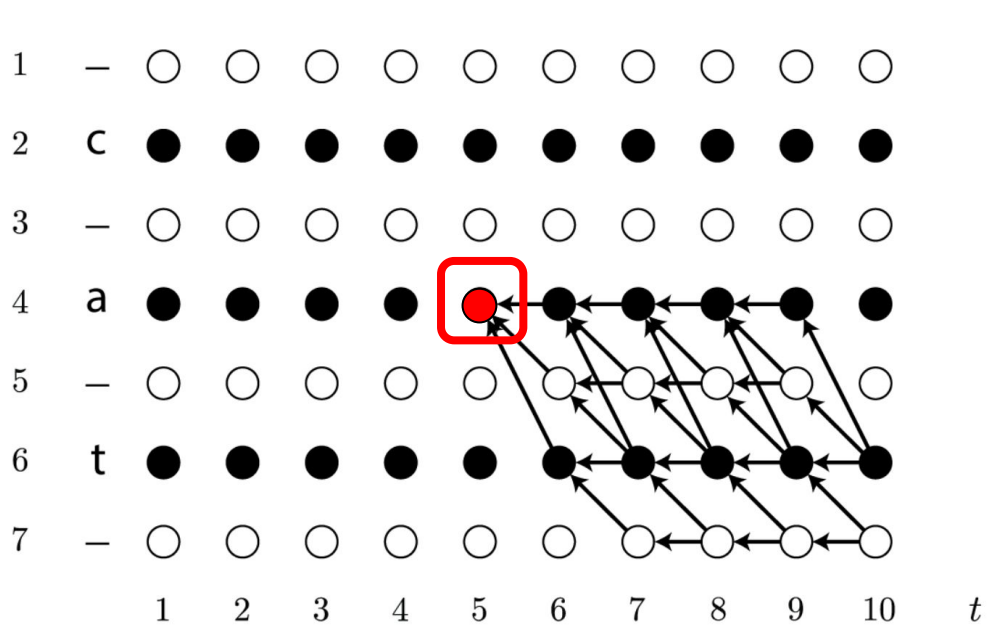
CTC Backward Probabilities

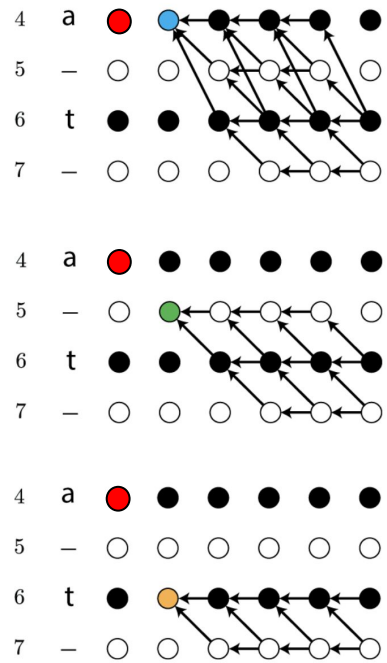
$$\beta(t,r) = P(s_{t+1} \in succ(S_r), succ(S_r), ..., S_{K-1}|X) = \sum_{q:S_q \in succ(S_r)} \beta(t+1,q) y_{t+1}^{S_q}$$

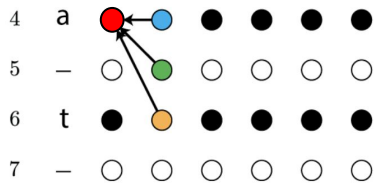


CTC Backward Probabilities

$$\beta(t,r) = P(s_{t+1} \in succ(S_r), succ(S_r), ..., S_{K-1}|X) = \sum_{q:S_q \in succ(S_r)} \beta(t+1,q) y_{t+1}^{S_q}$$



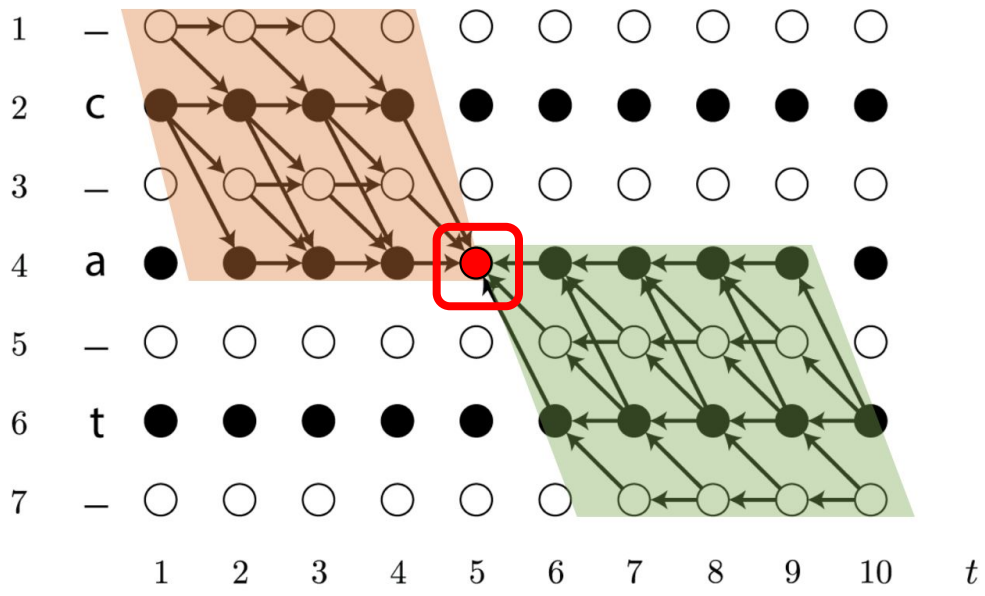




CTC Posterior Probabilities

- Represents probability of being in state **r** at time **t** given input sequence.

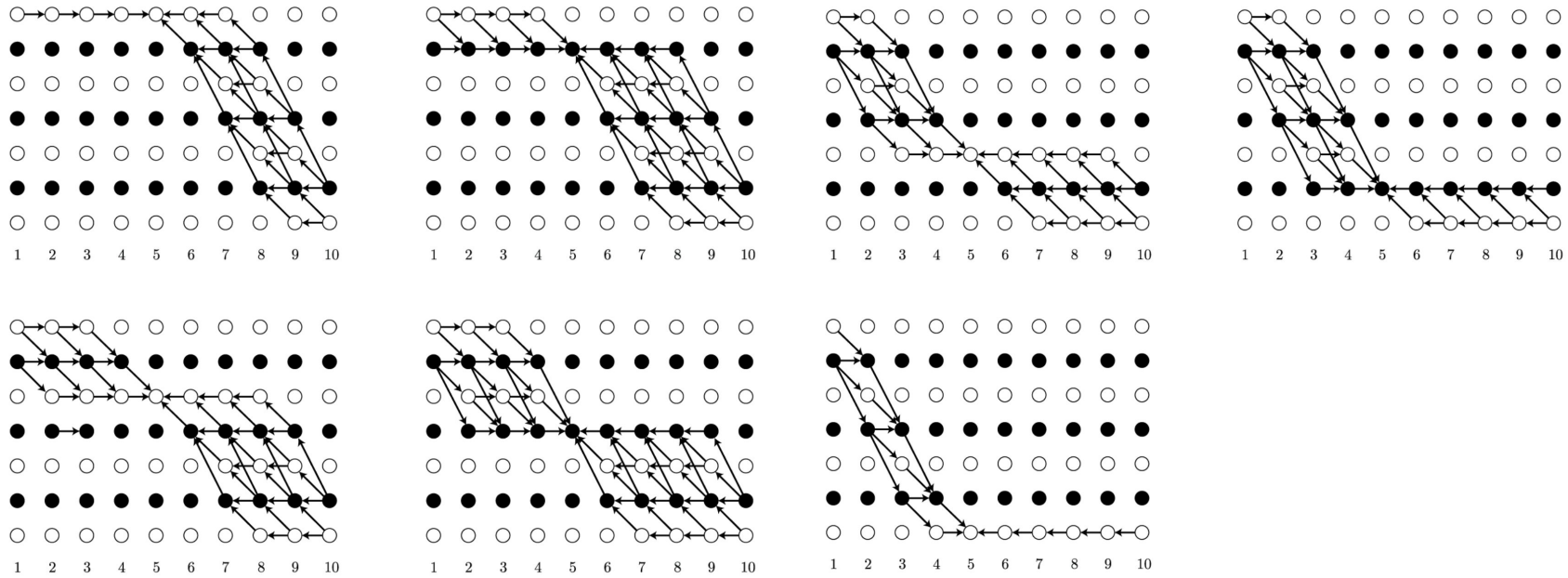
$$\gamma(t,r) = P(s_t = S_r | S, X) = \frac{\alpha(t,r)\beta(t,r)}{\sum_{r'} \alpha(t,r)\beta(t,r)}$$



CTC Posterior Probabilities

- Represents probability of being in state **r** at time **t** given input sequence.

$$\gamma(t,r) = P(s_t = S_r | S, X) = \frac{\alpha(t,r)\beta(t,r)}{\sum_{r'} \alpha(t,r)\beta(t,r)}$$



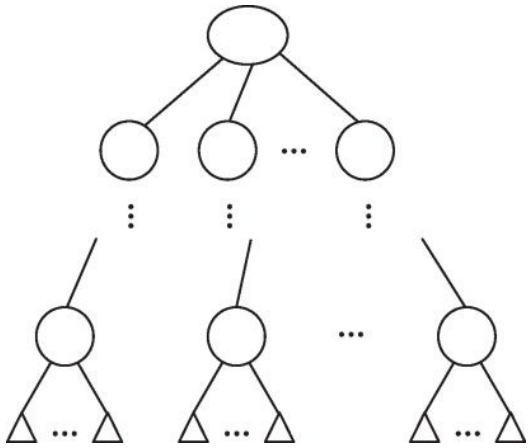
Inference

Q: What’s different at inference?

Q: What’s different at inference?

A: No target

A: No sequence “rules”



What are our options?

- Greedy Search
- Exhaustive Search
- Beam Search

Greedy Search (in the context of words)

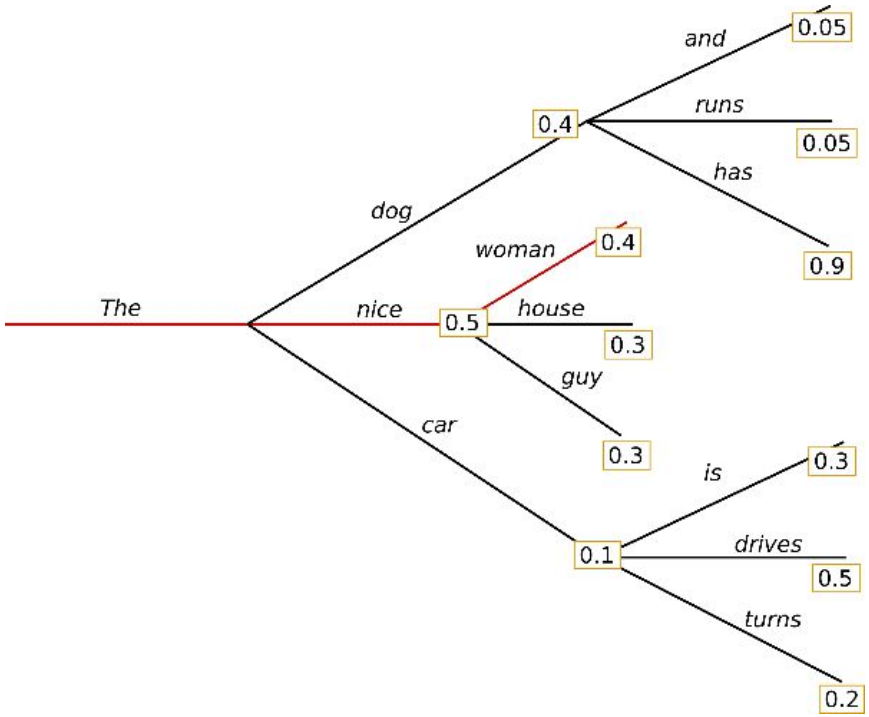
Given the previous (apriori), get the highest probability next word.

So $w_t = argmax_w P(w \mid w_{w1:t-1})$

Where w_t is our next sequence (in this case its a word).

Main issue:

- It does not represent the highest probability (best candidate sequence).



Greedy Search in Sequence to Sequence Inferencing

Greedy Search

Y_probs

	T = 0	T = 1	T = 2	T = 3
-	0.140	0.257	0.248	0.149
A	0.391	0.096	0.402	0.336
B	0.197	0.341	0.267	0.358
C	0.271	0.305	0.083	0.157

DECODED STRING

?

Greedy Search

Y_probs

	T = 0
-	0.140
A	**0.391**
B	0.197
C	0.271

A

Greedy Search

Y_probs

	T = 0	T = 1
-	0.140	0.257
A	**0.391**	0.096
B	0.197	**0.341**
C	0.271	0.305
	A	B

Greedy Search

Y_probs

	T = 0	T = 1	T = 2
-	0.140	0.257	0.248
A	**0.391**	0.096	**0.402**
B	0.197	**0.341**	0.267
C	0.271	0.305	0.083
	A	B	A

Greedy Search

Y_probs

	T = 0	T = 1	T = 2	T = 3
-	0.140	0.257	0.248	0.149
A	**0.391**	0.096	**0.402**	0.336
B	0.197	**0.341**	0.267	**0.358**
C	0.271	0.305	0.083	0.157
	A	B	A	B

Greedy Search

Y_probs

	T = 0	T = 1	T = 2	T = 3
-	0.140	0.257	0.248	0.149
A	0.391	0.096	0.402	0.336
B	0.197	0.341	0.267	0.358
C	0.271	0.305	0.083	0.157

DECODED STRING

A B A B

0.391*0.341*0.402*0.358 = 0.0191884642

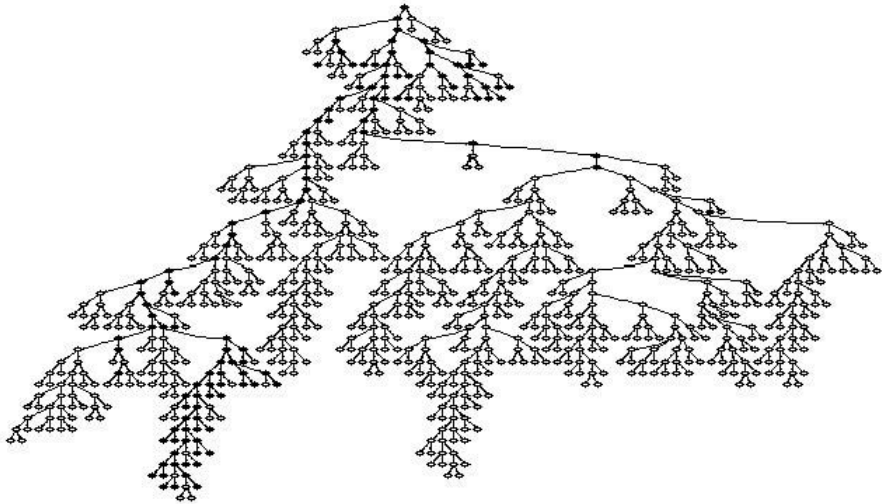
The other side (exhaustive testing).

We can exhaustively look through the sequence.

This would be a sure fire method in finding the highest probability sequence.

The main issue is

- It’s computational time is Massive (exponential).



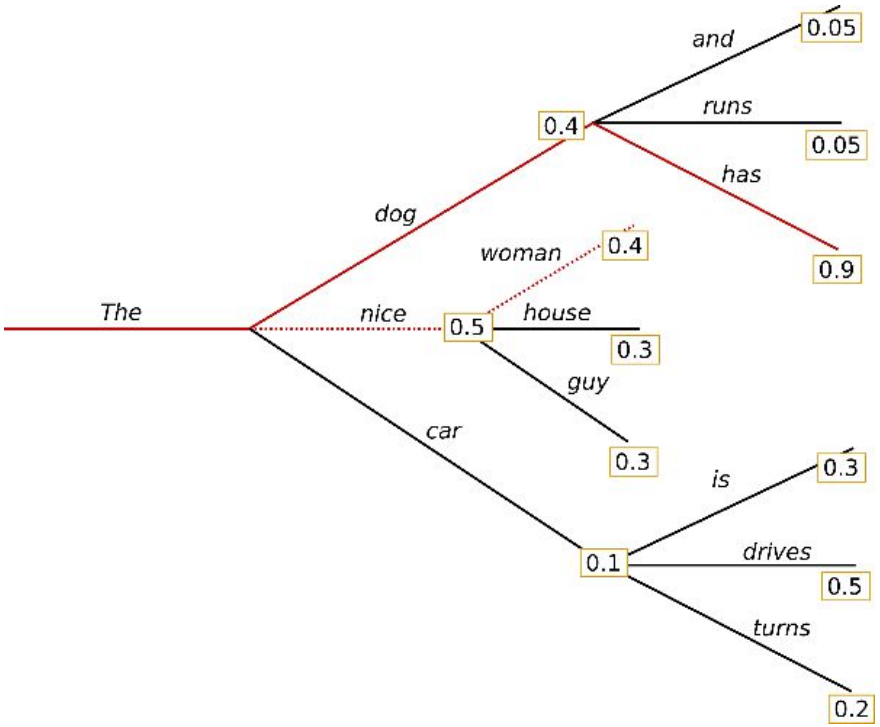
Non realistic example of all possible combinations.

The Middle ground: Beam Search

Beam search is a combination of greedy search and exhaustively searching.

Were we maintain the top k best candidates (which are our beam size).

These beams are calculate by grabbing what we've seen so far, and multiplying it by the predicted (future sequences). Similar to what we've seen in CTC loss.



Beam size of 2 (for words).

Beam Search in Sequence to Sequence Inferencing

Beam Search

Y_probs

	T = 0	T = 1	T = 2	T = 3
-	0.140	0.257	0.248	0.149
A	0.391	0.096	0.402	0.336
B	0.197	0.341	0.267	0.358
C	0.271	0.305	0.083	0.157

DECODED STRING

?

Parameters

Seq Len	4
Symbol set	{‘-’, ‘A’, ‘B’, ‘C’}
Beam Width	3

Beam Search

BEAM WIDTH = 3

	T = 0
-	0.140
A	0.391
B	0.197
C	0.271

Scores

Possible Paths	Calculate Score	Score
-	1*0.140	0.140
A	1*0.391	0.391
B	1*0.197	0.197
C	1*0.271	0.271

Old

Best Paths	Score
–	–
–	–
–	–

Beam Search

BEAM WIDTH = 3

	T = 0
-	0.140
A	0.391
B	0.197
C	0.271

Scores

Possible Paths	Calculate Score	Score
-	1*0.140	0.140
A	1*0.391	0.391
B	1*0.197	0.197
C	1*0.271	0.271

Old

Best Paths	Score
–	–
–	–
–	–

New

Best Paths	Score
A	0.391
B	0.197
C	0.271

Beam Search

BEAM WIDTH = 3

	T = 1
-	0.257
A	0.096
B	0.341
C	0.305

Scores

Possible Paths	Calculate Score	Score
A-	0.391*0.257	0.10048700
B-	0.197*0.257	0.05062900
C-	0.271*0.257	0.06964700
AA -> A	0.391*0.096	0.03753600
AB	0.391*0.341	0.13333100
AC	0.391*0.305	0.11925500
BA	0.197*0.096	0.01891200
BB -> B	0.197*0.341	0.06717700
BC	0.197*0.305	0.06008500
CA	0.271*0.096	0.02601600
CB	0.271*0.341	0.09241100
CC -> C	0.271*0.305	0.08265500

Old

Best Paths	Score
A	0.391
B	0.197
C	0.271

Beam Search

BEAM WIDTH = 3

	T = 1
-	0.257
A	0.096
B	0.341
C	0.305

Scores

Possible Paths	Calculate Score	Score
A-	0.391*0.257	0.10048700
B-	0.197*0.257	0.05062900
C-	0.271*0.257	0.06964700
AA -> A	0.391*0.096	0.03753600
AB	0.391*0.341	0.13333100
AC	0.391*0.305	0.11925500
BA	0.197*0.096	0.01891200
BB -> B	0.197*0.341	0.06717700
BC	0.197*0.305	0.06008500
CA	0.271*0.096	0.02601600
CB	0.271*0.341	0.09241100
CC -> C	0.271*0.305	0.08265500

Old

Best Paths	Score
A	0.391
B	0.197
C	0.271

New

Best Paths	Score
A-	0.10048700
AB	0.13333100
AC	0.11925500

Beam Search

BEAM WIDTH = 3

	T = 2
-	0.248
A	0.402
B	0.267
C	0.083

Scores

Possible Paths	Calculate Score	Score
A--	0.100487*0.248	0.0249207760
AB-	0.133331*0.248	0.0330660880
AC-	0.119255*0.248	0.0295752400
A-A -> AA	0.100487*0.402	0.0403957740
ABA	0.133331*0.402	0.0535990620
ACA	0.119255*0.402	0.0479405100
A-B -> AB	0.100487*0.267	0.0268300290
ABB -> AB	0.133331*0.267	0.0355993770
ACB	0.119255*0.267	0.0318410850
A-C -> AC	0.100487*0.083	0.0083404210
ABC	0.133331*0.083	0.0110664730
ACC -> AC	0.119255*0.083	0.0098981650

Old

Best Paths	Score
A-	0.10048700
AB	0.13333100
AC	0.11925500

Beam Search

BEAM WIDTH = 3

	T = 2
-	0.248
A	0.402
B	0.267
C	0.083

Scores

Possible Paths	Calculate Score	Score
A--	0.100487*0.248	0.0249207760
AB-	0.133331*0.248	0.0330660880
AC-	0.119255*0.248	0.0295752400
A-A -> AA	0.100487*0.402	0.0403957740
ABA	0.133331*0.402	0.0535990620
ACA	0.119255*0.402	0.0479405100
A-B -> AB	0.100487*0.267	0.0268300290
ABB -> AB	0.133331*0.267	0.0355993770
ACB	0.119255*0.267	0.0318410850
A-C -> AC	0.100487*0.083	0.0083404210
ABC	0.133331*0.083	0.0110664730
ACC -> AC	0.119255*0.083	0.0098981650

Old

Best Paths	Score
A-	0.10048700
AB	0.13333100
AC	0.11925500

New

Best Paths	Score
ABA	0.0535990620
ACA	0.0479405100
AB	0.0624294060

Beam Search

BEAM WIDTH = 3

	T = 3
-	0.149
A	0.336
B	0.358
C	0.157

Scores

Possible Paths	Calculate Score	Score
ABA- -> ABA	0.0535990620*0.149	0.0079862602380
ACA- -> ACA	0.0479405100*0.149	0.0071431359900
AB- -> AB	0.0624294060*0.149	0.0093019814940
ABAA -> ABA	0.0535990620*0.336	0.0180092848320
ACAA -> ACA	0.0479405100*0.336	0.0161080113600
ABA	0.0624294060*0.336	0.0209762804160
ABAB	0.0535990620*0.358	0.0191884641960
ACAB	0.0479405100*0.358	0.0171627025800
ABB -> AB	0.0624294060*0.358	0.0223497273480
ABAC	0.0535990620*0.157	0.0084150527340
ACAC	0.0479405100*0.157	0.0075266600700
ABC	0.0624294060*0.157	0.0098014167420

Old

Best Paths	Score
ABA	0.0535990620
ACA	0.0479405100
AB	0.0624294060

Beam Search

BEAM WIDTH = 3

	T = 3
-	0.149
A	0.336
B	0.358
C	0.157

Scores

Possible Paths	Calculate Score	Score
ABA- -> ABA	0.0535990620*0.149	0.0079862602380
ACA- -> ACA	0.0479405100*0.149	0.0071431359900
AB- -> AB	0.0624294060*0.149	0.0093019814940
ABAA -> ABA	0.0535990620*0.336	0.0180092848320
ACAA -> ACA	0.0479405100*0.336	0.0161080113600
ABA	0.0624294060*0.336	0.0209762804160
ABAB	0.0535990620*0.358	0.0191884641960
ACAB	0.0479405100*0.358	0.0171627025800
ABB -> AB	0.0624294060*0.358	0.0223497273480
ABAC	0.0535990620*0.157	0.0084150527340
ACAC	0.0479405100*0.157	0.0075266600700
ABC	0.0624294060*0.157	0.0098014167420

Old

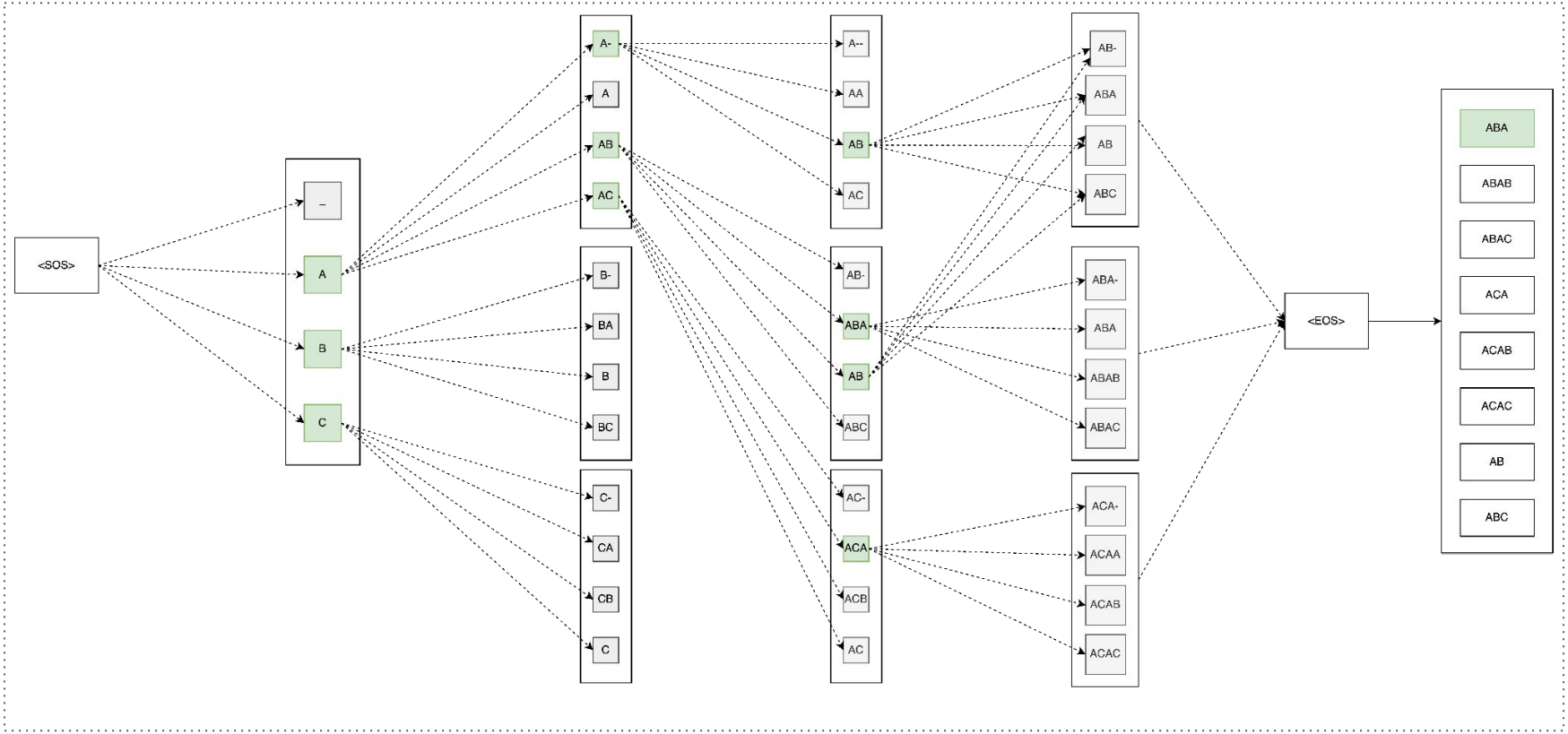
Best Paths	Score
ABA	0.0535990620
ACA	0.0479405100
AB	0.0624294060

New

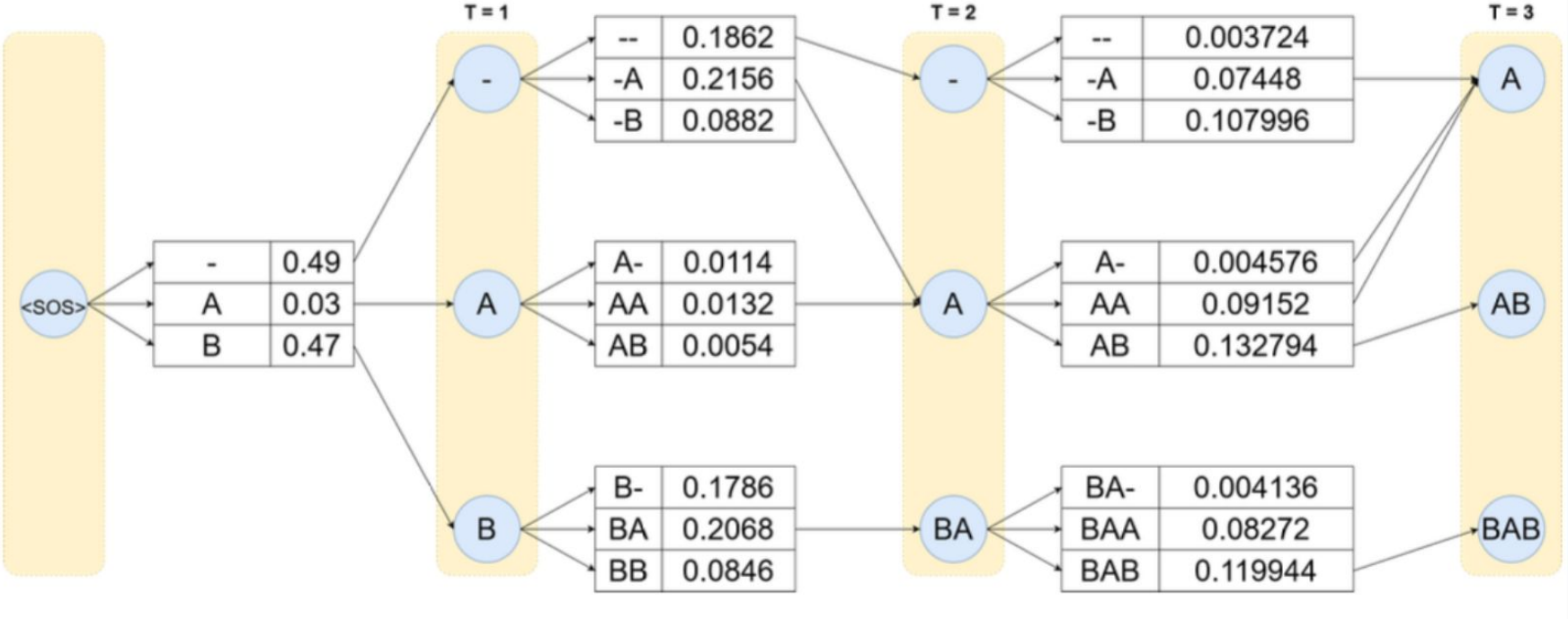
Best Paths	Score
ABA	0.04697182548
AB	0.031651708842
ACA	0.02325114735

Beam Search

BEAM WIDTH = 3



Another representation (different set of probabilities)



Beam Search

Remember, when extending a path with a new symbol, you'll encounter three scenarios:

1. The new symbol is the same as the last symbol on the path.
2. The last symbol of the path is blank.
3. The last symbol of the path is different from the new symbol and is not blank.

Efficient Beam Search:

```
Input: SymbolSets, y_probs, BeamWidth
Output: BestPath, MergedPathScores

    0. Initialize:
        1. BestPaths with a blank symbol path with a score of 1.0.
        2. TempBestPaths as an empty dictionary.
    1. For each timestep in y_probs:
        1. Extract the current symbol probabilities.
        2. For each path, score in BestPaths limited by BeamWidth:
            1. For each new symbol in the current symbol probabilities:
                1. Based on the last symbol of the path, determine the new path.
                2. Update the score for the new path in TempBestPaths.

        3. Update BestPaths with TempBestPaths.
        4. Clear TempBestPaths.

    2. Initialize MergedPathScores as an empty dictionary.

    3. For each path, score in BestPaths:
        1. Remove the ending blank symbol from the path.
        2. Update the score for the translated path in MergedPathScores.
        3. Update the BestPath and BestScore if the score is better.

    4. Return BestPath and MergedPathScores.
```

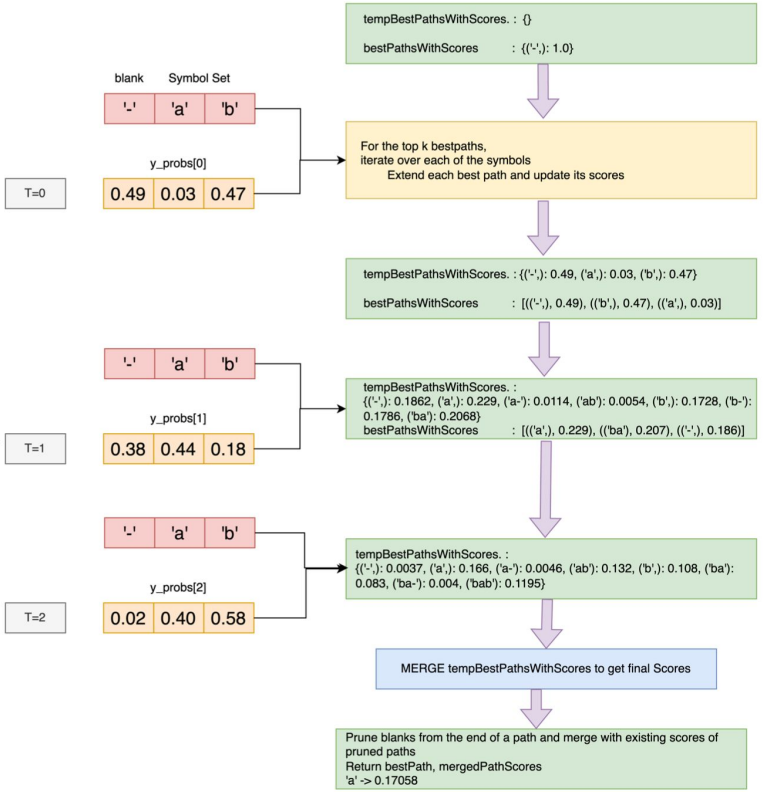


Figure 20: Efficient Beam Search procedure